Opportunistic ZGC: Leveraging Idle Cores for More Effective Concurrent Garbage Collection

Jacob Malloy
jmalloy1@utk.edu
University of Tennessee
Knoxville, Tennessee, USA

Michael R. Jantz✉
mrjantz@utk.edu
University of Tennessee
Knoxville, Tennessee, USA

Terry Jones
trjones@ornl.gov
Oak Ridge National Laboratory
Oak Ridge, Tennessee, USA

arXiv:2609.15558v1 [cs.PL] 14 Sep 2026

Abstract

Managed language runtimes often provide concurrent garbage collectors so that latency-critical applications with large working sets can keep running while most collection work proceeds in the background. ZGC is a production-quality, generational, concurrent collector in OpenJDK with sub-millisecond pause times. While ZGC is designed to run concurrently, frequent and excessive collections with ZGC can still slow the mutators due to synchronization costs and interference with shared computing resources. Hence, the ZGC scheduler is conservative by default, and in most cases, will grow the heap toward the maximum allowed before scheduling a collection. While this approach minimizes collection effort, it can be wasteful, or even harmful, if the maximum heap size is not well tuned to the actual working set.

We propose Opportunistic ZGC (OppZGC), a feedback-directed ZGC scheduling policy that constrains the heap *dynamically* and *automatically*, without per-application tuning. OppZGC identifies periods when CPU cores are underutilized and leverages them for concurrent collection with ZGC. We describe the design and implementation of OppZGC in OpenJDK's HotSpot Java VM and evaluate it with standard and latency-sensitive benchmarks from DaCapo Chopin and SPECjbb. OppZGC limits heap usage when there is CPU capacity sufficient for additional collections, and avoids scheduling extra collections when they would substantially degrade performance. Overall, it reduces maximum heap usage for our DaCapo benchmarks by between 61% and 90%, on average, depending on configuration, with minimal impact on throughput and request latency compared to default ZGC.

CCS Concepts: • **Software and its engineering → Garbage collection**; *Runtime environments.*

1 Introduction

Many of today's most popular programming languages, including Java, Go, Python, and JavaScript, manage memory automatically with garbage collection (GC). Runtimes for these languages implement a variety of GC strategies to meet the needs of a broad spectrum of applications and usage scenarios. For example, recent versions of the HotSpot Java Virtual Machine (JVM) (from OpenJDK) provide a range of GC options with tradeoffs in throughput and pause times. ParallelGC, HotSpot's default prior to JDK 9, and G1GC, which is the default since then, are both generational, compacting collectors with multi-threaded, mostly stop-the-world collection. While these collectors achieve excellent throughput, they can incur long pauses for large heaps, which may be unacceptable for latency-critical applications.

To support such applications, HotSpot introduced ZGC in JDK 11, a fully concurrent collector with sub-millisecond pause times practically independent of the size of the heap [13]. In general, ZGC sacrifices some throughput to enable more fine-grained sharing of computing resources. It relies on load and store barriers (with colored pointers) to implement incremental marking and relocation of program data objects.

Since allocation and collection can occur simultaneously, ZGC is vulnerable to scenarios where the allocation rate outpaces the collection rate and exhausts the available heap capacity, forcing the mutators to stall. Even if heap capacity is sufficient to avoid such *allocation stalls*, frequent and excessive collection with ZGC can still harm performance by causing frequent synchronization pauses, increasing the rate of slow paths at barrier instructions, and stealing or interfering with shared computing resources.

To avoid unnecessary work and potential slowdowns, the default ZGC scheduler is relatively conservative. In most cases, it grows the heap toward the maximum allowed by the JVM instance and defers collection until runtime heuristics predict that delaying further would risk an allocation stall. While this approach minimizes GC effort, it can be wasteful, or even harmful, if the working set of the application is much smaller than the maximum heap size. Besides monopolizing memory resources that may be better used by other applications, it can also hurt performance by increasing paging overheads and reducing locality in the heap.

ZGC should thus employ a maximum heap size that is large enough to prevent allocation stalls and mutator interference, but not so large that it wastes memory or harms locality. Striking the proper balance can be notoriously difficult, and the community currently lacks approaches for tuning ZGC scheduling parameters automatically. Best practice is empirical testing with JVM tunables such as `-Xmx` and `-XX:SoftMaxHeapSize` [30], which requires manual effort and leaves deployments exposed when production demands diverge from tuned expectations.

We address these gaps with Opportunistic ZGC (OppZGC), a feedback-directed GC scheduling policy that *dynamically*

and *automatically* constrains the heap by identifying periods when CPU cores are underutilized and leveraging them for concurrent collection. It relies on two key observations. First, many execution scenarios have frequent periods where one or more computing cores are underutilized (e.g., due to over-provisioning or imperfect scaling). Second, the current ZGC scheduler is often too conservative and wasteful of memory resources. While more frequent collection can harm mutator performance, a more aggressive scheduler can limit these costs and effectively constrain heap occupancy as long as additional GC cycles: 1) are metered by growth in the heap, and 2) only run on otherwise idle CPU.

We have built OppZGC by extending the open source HotSpot JVM in the Java Development Kit (JDK) (jdk-24+2, mainline commit 50bed6c) [17]. The design is straightforward; it extends the ZGC scheduler with two additional rules for inducing collection cycles when sufficient CPU capacity is available. However, our evaluation, which includes standard benchmarks from DaCapo Chopin v. 23.11-MR2 [5] as well as the memory-intensive and latency-sensitive SPECjbb [20], demonstrates that this approach effectively constrains memory usage for applications that use ZGC, without increasing the latency of critical operations or reducing application throughput, in almost every case.

This work makes the following important contributions.

- It describes the design of Opportunistic ZGC: a ZGC scheduling policy that schedules collections when the heap has grown beyond a configurable multiple of its recent occupancy *and* there is enough computing capacity available to complete the cycle.
- It evaluates OppZGC on two x86-64 platforms: an Intel-based server and an AMD-based desktop machine. OppZGC reduces memory usage substantially compared to baseline ZGC with minimal impact throughput and latency. On the Intel platform, it reduces maximum heap usage for the DaCapo benchmarks by between 61% and 90%, on average, depending on the configuration. For SPECjbb, it reduces heap usage by up to 21%, with no harm to maximum or critical jOPS in large memory configurations.
- It evaluates OppZGC with varying amounts of CPU capacity available to the Java process. OppZGC effectively adapts its scheduling policy to CPU-constrained environments. With only two cores available, OppZGC still exhibits similar throughput and latency performance as default ZGC and reduces maximum heap usage for the DaCapo default and large input sets by 63% and 39%, respectively.

2 Concurrent Collection with ZGC

This section describes the design and implementation of ZGC from JDK 24, which is the basis for OppZGC, as well as current options for tuning application heap sizes with ZGC.

2.1 ZGC Design and Implementation

ZGC is a mark-evacuate concurrent collector built for parallel architectures. Almost all of its collection work, including marking and relocation, is multi-threaded and performed while mutators are running. ZGC's heap is divided into regions, called ZPages, which are based on size classes. New objects are bump-pointer allocated into the ZPage of the appropriate size class. ZGC periodically defragments the heap by copying surviving objects from sparse ZPages, which are then freed. This design allows ZGC to control collection costs because it can choose to relocate objects on only a fraction or none of the ZPages, depending on need. Generational ZGC [21], introduced in JDK 21 and now the only mode available in JDK 24, additionally partitions ZPages based on the age of their data and performs separate cycles for minor (young-only) and major (young and old) collections.

2.1.1 Colored Pointers and Barrier Instructions.

To ensure that the concurrent mutator and GC threads always see valid pointers, ZGC uses colored pointers and barrier instructions. Object references in the ZGC heap are always 64 bits: address bits plus meta bits describing the pointer's current "color". ZGC maintains global status bits, which mutators check against a pointer's color to determine if the pointer is "good" (i.e., valid and usable with no extra work required) in the current execution context, and updates them as the cycle advances. In generational ZGC, these bits can also distinguish "store-good" from "load-good" pointers, since a pointer that is valid to dereference may still require GC work before it can be overwritten.

Since collector threads can relocate program objects while the mutator threads are running, ZGC inserts load barriers at pointer read instructions to avoid using or following stale pointers. If the color of the pointer being read is load-good, the barrier fast path simply strips the pointer color and allows the mutator to use the colorless pointer. Otherwise, the barrier slow path determines if the object is (or is about to be) relocated, and if so, finds (or decides) the new address of the object. The pointer is then *healed* by writing the updated, good-colored pointer back to the field from which it was loaded. In this way, subsequent loads of the same field will use the fast path until the ZGC cycle updates the status bits.

ZGC employs store barriers at pointer writes to add color to colorless pointers held on the stack or in registers before they are written to the heap. Store barriers serve two other purposes: 1) maintaining the *remembered set* (the set of old generation fields that may contain references to young objects), which is necessary to avoid scanning the old generation during minor collections, and 2) implementing its snapshot-at-the-beginning (SATB) approach for marking, which ensures a consistent view of the heap if a mutator overwrites a pointer during marking [28]. ZGC fuses the checks for both cases into a single fast-path test at each

Type	Name	Fires When
Major GC Rules	`major_timer`	Time since last major GC exceeds `ZCollectionIntervalMajor`. Disabled by default.
	`major_warmup`	Heap usage crosses threshold percentages of the soft maximum. Startup rule only.
	`major_proactive`	Enough time or growth has passed that the throughput cost of major GC is acceptable.
	`major_allocation_rate`	An already-warranted minor collection should be upgraded to a major collection, typically because the estimated cost efficiency of major collection is higher than minor collection.
Minor GC Rules	`minor_timer`	Time since last minor GC exceeds `ZCollectionIntervalMinor`. Disabled by default.
	`minor_allocation_rate`	Predicted allocation rate implies heap will be exhausted before a young collection finishes.
	`minor_high_usage`	Free memory drops below 5% of soft max. Backstop for cases with low allocation rate.

Table 1. ZDirector rules for initiating major and minor GC.

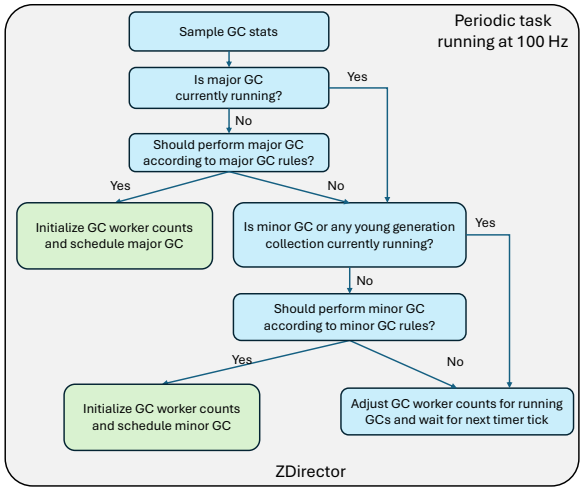


Figure 1. ZDirector decisions and control flow. For simpler presentation, this figure omits cases where an already-warranted minor GC may be upgraded to major GC.

pointer write. If either check fails, the slow path performs the required work and recolors the field.

2.1.2 Scheduling ZGC Collections.

While certain events, such as allocation stalls and explicit GC requests from the application, can initiate ZGC collections, minor and major GCs are primarily invoked from an asynchronous periodic task, called ZDirector. Figure 1 diagrams the control flow and major decisions made by ZDirector.

ZDirector operates at a fixed frequency of 100 Hz. At each timer tick, it collects statistics on the allocation rate (averaged and smoothed over recent intervals), current heap usage, and per-generation accounting such as timing stats and bytes reclaimed by previous collections. It then uses this information to check a series of rules that determine whether or not to initiate a GC cycle. If a major collection is not currently running, it checks the rules for running a major collection. Then, if no major GC is scheduled or running, it checks if minor GC, or the young cycle of a major GC, is already running, and if not, it checks the rules for minor collections. If either path fires, ZDirector sets the initial number of GC workers based on its sampled statistics and schedules a minor or major GC to start in a separate thread. Otherwise, it may still adjust the worker count for active collection cycles before sleeping until the next timer tick.

Table 1 presents the ZDirector rules. The timer rules are unused by default and `major_warmup` fires at most three times during startup. The other rules are conservative by design. `major_proactive` waits until the heap grows by at least 10% or 5 minutes have passed since the last major GC, and assumes 50% mutator throughput reduction during collection. `minor_high_usage` only fires when free memory is below 5% of the soft maximum allowed and mainly serves as a backstop for when the allocation rate rules do not fire because new allocations have slowed to a trickle.

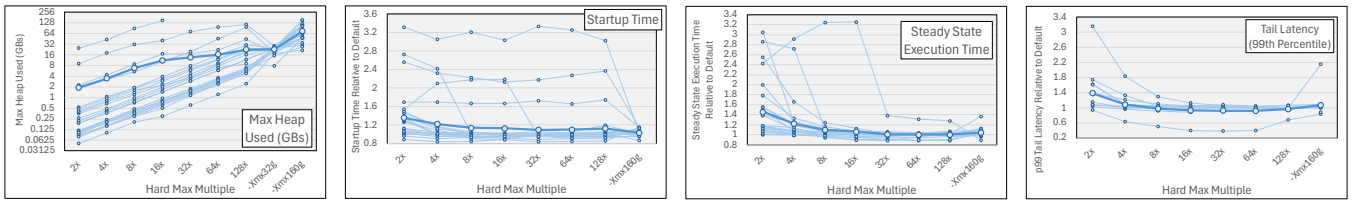
In practice, most collections are triggered by the *minor_allocation_rate* rule. *major_allocation_rate* is checked only after a minor rule fires, and may upgrade the impending minor GC to a major GC. Since these rules defer collection until new allocations are expected to exhaust free memory within a short time, the ultimate effect of this strategy is to grow the heap until it is near the soft maximum allowed.

2.1.3 Young and Old Collection Cycles.

Generational ZGC has two types of collection cycles: minor GCs perform only the young collection cycle, while major GCs perform the young collection followed by the old. Each cycle consists of a small number of sub-millisecond STW pauses and several concurrent phases. Appendix A describes the cycles in more detail. Yang and Wrigstad [25] provide a comprehensive description of the underlying collection mechanisms, though for an earlier, single-generation version of ZGC (JDK 15)

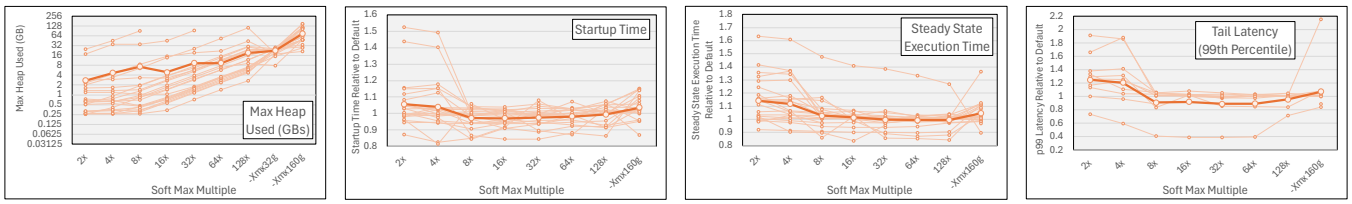
2.2 Controlling the Frequency of ZGC

Beyond explicit calls (e.g., `System.gc()`) and the timer rules above, users primarily control ZGC frequency by limiting the size of the heap. Java provides two command line options for this purpose: `-Xmx`, which is a hard limit, and `-XX:SoftMaxHeapSize`, which is a soft limit that ZGC strives to stay within. The hard max defaults to 25% of physical memory, up to a maximum of 32 GB, and the soft max defaults to the hard max. ZDirector proactively invokes collections to try to keep the heap occupancy under the soft max. If a new allocation would breach the hard max, the mutators will stall until a collection has freed enough space (or, if none can be freed, the JVM reports out of memory).



(a) Max heap used (GB, log scale). **(b)** Startup time. **(c)** Steady-state execution time. **(d)** Tail latency (99th percentile).

Figure 2. DaCapo benchmarks with hard max (-Xmx) set to different multiples of the minimum heap size.



(a) Max heap used (GB, log scale). **(b)** Startup time. **(c)** Steady-state execution time. **(d)** Tail latency (99th percentile).

Figure 3. DaCapo benchmarks with soft max (-XX:SoftMaxHeapSize) set to different multiples of the minimum heap size.

To illustrate the effect of these options with ZGC, we measured the minimum heap sizes of of each of our DaCapo benchmarks with default ZGC (both default and large inputs, using the methodology described in [5]), and ran each workload in isolation on our Intel platform with hard max or soft max set to one of a series of multiples of its minimum size.[1] We compare against default configurations with the hard maximum set to 32 GB (the Java-selected default on our server platform) and with the hard maximum set to 160 GB (most of the 192 GB available). The soft max experiments all use a hard max of 192 GB, which is never reached.

Figures 2 and 3 show the maximum heap usage and performance, in terms of startup time, steady-state execution time, and, for the ten benchmarks that report latency statistics, the p99 tail latency, for each hard max and soft max configuration. In each figure, each line represents a single benchmark, and the thicker, darker line shows the average. Aside from the heap usage results, all results are shown relative to the baseline ZGC configuration with -Xmx set to 32 GB. Some lines are incomplete because some benchmarks crash if the maximum heap size is larger than the available memory.

Observe that both hard max and soft max can effectively constrain heap memory utilization. At lower multiples, hard max uses significantly less memory than soft max, but these savings come at a steep performance price. For example, with the limits set at 2× the minimum, hard max is about 30% slower, on average, in both startup and steady-state execution time and its average tail latency is about 10% worse than soft max. These degradations are mainly the result of frequent *allocation stalls*, which pause the mutators until a collection can free up enough space to satisfy a new object.

While restricting the soft maximum also enables substantial memory savings, this approach exhibits much less performance downside. The most aggressive soft max configuration (2×) uses only 12% of the heap memory used by the default `-Xmx32g` configuration with startup and steady-state execution times only 6% and 13% worse, on average. Moreover, the 16× soft max configuration reduces memory usage by more than 77% with almost no performance downside.

Furthermore, some workloads actually perform better with a tighter hard or soft maximum than with a mostly unrestricted heap. With soft max set to 8× the minimum, `spring` with its default input reduces startup and steady-state time by 13% and 15% and tail latency by 17%. These gains are primarily driven by reduced paging costs. For `spring` specifically, the default configuration incurs 19× as many page faults and 21% more DTLB misses.

These trends also hold under varying CPU constraints. Repeating the soft max sweep with the cores restricted to 8, 4, or 2 CPUs (see Figure 9 in Appendix C.1.1) yields broadly similar relative performance at most multiples, but the most aggressive configurations degrade further. With only two cores and soft max at 2×, startup and steady-state slowdowns grow to 15% and 20%, and tail latency degrades by over 70%.

While tuning the hard or soft maximum can yield substantial benefits, this approach is infeasible in many real-world scenarios. For many applications, heap usage is either poorly understood or unpredictable in a way that makes it difficult to select an appropriate maximum. Furthermore, since GC workers can steal resources from mutators, GC effort must also be balanced against limits on CPU time, increased barrier slow path rate, and potential interference in shared caches. For these reasons, HotSpot developers recommend that ZGC users choose heap limits that are large enough to

[1]The minimum heap sizes we measured are listed in Table 4 of Appendix B.

Metric	Description	How It Is Computed	Example
idle_cores	Currently idle CPU capacity (in cores)	Active processor count − CPU consumed over the previous δ time units	If δ = 100 ms and if 10 of 12 cores are utilized at 90% over the previous 100 ms, then: idle_cores = $12 - (10 \times 0.9) = 3$.
gc_duration	Wall clock duration of a GC cycle	Serial GC time + $\frac{\text{Parallelizable GC time}}{\text{GC Worker Count}}$	If serial GC time = 100 ms and parallelizable GC time = 400 ms, and there are 4 GC workers, then: gc_duration = $100\,\text{ms} + \frac{400\,\text{ms}}{4} = 200\,\text{ms}$.
gc_cpu_cost	Total CPU cost (in CPU time) of a GC cycle	Serial GC time + Parallelizable GC time	If serial GC time = 100 ms and parallelizable GC time = 400 ms, then: gc_cpu_cost = 100 ms + 400 ms = 500 ms.

Table 2. Metrics used to implement major GC and minor GC rules for OppZGC. Assuming sufficient growth in heap occupancy, the example will trigger GC because it passes Condition 1 (i.e., $3 \times 200\,\text{ms} = 600\,\text{ms} \geq 500\,\text{ms}$).

avoid potential performance problems, even if the chosen size is likely to be wasteful of memory capacity [18, 31].

3 Opportunistic ZGC: Leveraging Idle Cores for More Effective Concurrent GC

The previous section shows there is a need for an automated approach that effectively constrains the ZGC heap, without slowing the mutators with interference from ZGC workers. OppZGC fills this gap with a GC scheduling policy that runs GC workers in otherwise idle CPU capacity.

3.1 OppZGC Collection Rules

OppZGC extends ZDirector with two new rules, `major_opportunistic` and `minor_opportunistic`, that fire when the runtime estimates there is enough idle CPU to complete the cycle. Each rule is checked only after the other rules for that GC type have all been checked and failed to fire.

Running GC in every possible window, even if there is idle CPU capacity, could reduce mutator throughput by increasing the frequency of STW pauses and barrier slow paths. Hence, each rule first checks if the relevant generation's heap occupancy has grown enough since the previous opportunistic collection to justify another cycle. Specifically, growth in heap occupancy since that collection, normalized by the amount of garbage it reclaimed, must exceed a configurable *growth ratio* α. α values below 1.0 use free computing capacity to exert downward pressure on the heap, while α values at or slightly above 1.0 cap heap growth to be at or near what was recently reclaimed. During warmup, this check is skipped if there are no previous opportunistic collections.

If the heap has grown beyond the α-based threshold, the OppZGC rules then compute an estimate to predict if there is currently enough idle CPU capacity to run a major or minor GC cycle. For this estimate, OppZGC employs three runtime-derived metrics, which are shown in Table 2.

ZGC already maintains running averages of the serial and parallelizable times of previous cycles, and OppZGC uses these averages to estimate gc_duration and gc_cpu_cost. For idle_cores, it measures the CPU capacity (in cores) used over the past δ time units, where δ is a configurable multiple of the ZDirector timer tick, and subtracts that from the total cores available. We considered alternative strategies for estimating CPU utilization, including using a moving average to smooth bursty periods and discounting CPU usage by GC workers, but found no benefit to these approaches in initial testing.

After computing the necessary metrics, the OppZGC rules each check the condition:

$$\text{idle_cores} \times \text{gc_duration} \geq \text{gc_cpu_cost} \tag{1}$$

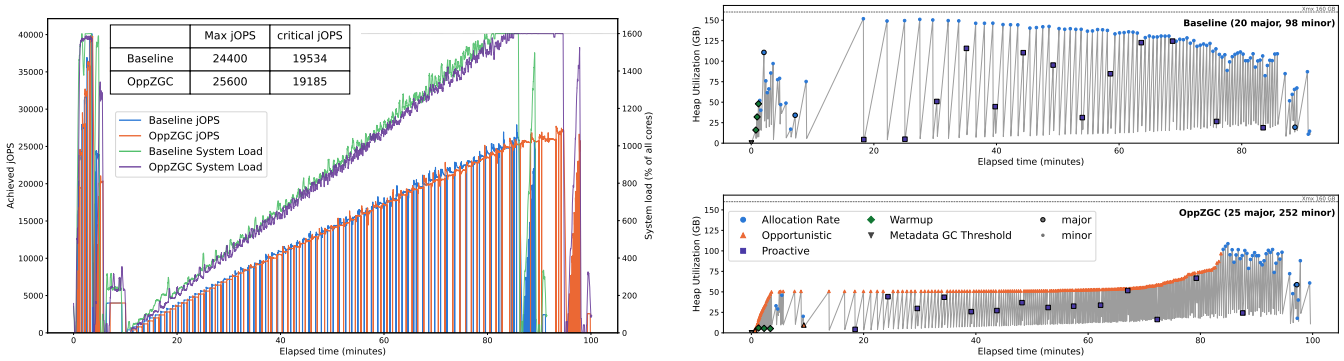
If the condition is met, then the rule initiates the appropriate GC cycle. Otherwise, no action is taken and ZDirector continues without invoking a GC cycle as shown in Figure 1.

Since past CPU utilization is not always predictive of future behavior, these rules may still decrease mutators' access to the CPU and harm application performance. However, even in these cases, the costs are still limited by two factors: 1) ZGC collections are mostly concurrent with only sub-millisecond pauses, and 2) OppZGC is self-modulating; that is, if the combined utilization of mutators and GC workers saturates the CPU, no new collections will be performed until there is idle capacity available (or another rule fires).

3.1.1 Implementation Notes and Details.

Our implementation employs the `getrusage` system call, which reports the CPU utilization of the calling process [15].[2] This approach is only appropriate for scenarios where the JVM runs on an otherwise idle machine. `/proc/schedstat`, for true system-wide CPU utilization, or the cgroup filesystem's `cpu.stat`, for per-cgroup CPU utilization, can be used for multi-process and multi-tenant usage scenarios. OppZGC does not scale the number of GC workers based on the idle CPU capacity, but rather uses the GC worker counts recommended by ZDirector, which are based on the size of the heap. During warmup, before estimates of gc_duration and gc_cpu_cost exist, the OppZGC rules check if there is at least one idle core available, rather than Condition 1. To avoid initiating multiple collections from stale information, OppZGC starts at most one GC cycle per δ period.

[2] `getrusage` supports μs resolution, but the actual resolution is platform-dependent. Both of our experimental platforms support μs resolution.



(a) System Load (CPU Utilization) and Performance in jOPS

(b) Collection Cycles with Baseline ZGC and OppZGC

Figure 4. Example Execution of SPECjbb with OppZGC.

3.2 Example execution with OppZGC

Figure 4 shows a detailed timeline of the total system load, memory usage, and collections during execution of the SPECjbb benchmark with two configurations: one with default ZGC and another with OppZGC enabled. For this comparison, both configurations set the hard maximum heap limit (`-Xmx`) to 160 GB, and the OppZGC configuration uses $\alpha = 1.0$ and $\delta = 20$ ms. A detailed description of the SPECjbb workload is provided in Section 4.2.2.

Observe that, during the initial high-intensity HBIR phase, there is still enough computing capacity available for frequent opportunistic collections to keep the heap occupancy below 60 GB. As system load ramps up during the RT-curve building phase, the baseline configuration quickly exhausts most of the remaining heap capacity, while OppZGC slows growth in the heap with additional minor collections. When the workload fully saturates the CPU, opportunistic collections stop because there is no longer CPU capacity available for extra collections. Other ZGC collection rules still fire during this phase, matching the behavior of baseline ZGC.

In this example, OppZGC causes relatively few major GCs. Since most data in SPECjbb dies before it is tenured, the old generation reaches its largest point early in the run, and the checks for additional growth in the old generation heap fail after this early point. In both configurations, most major cycles are caused by `major_proactive`. While the proactive rule has a similar goal to OppZGC, its heuristic is overly conservative because it assumes a 50% throughput drop during collection and does not account for idle computing capacity. Hence, the proactive rule alone does not limit heap usage nearly as much as OppZGC.

Overall, OppZGC constrains heap occupancy with only marginal impact on overall system load and performance. The maximum heap occupancy is reduced by about 21% compared to default ZGC. At the same time, SPECjbb peak throughput (max jOPS) and latency-constrained throughput (critical jOPS) are essentially the same in both configurations.

4 Experimental Setup

4.1 Experimental Platforms and Configuration

4.1.1 Testbed Hardware. Our primary evaluation platform contains a single Intel® Xeon® Gold 6246R CPU (codenamed Cascade Lake) with 16 physical compute cores and hyperthreading disabled. The cores all run a 3.4 GHz clock and share a 35.75 MB L3 cache. The processor includes a memory controller that services six channels, each of which is connected to one 32 GB, 2933 MT/s, DDR4 DIMM giving 192 GB of DDR4 SDRAM. To confirm the robustness of our results, we also evaluate OppZGC on an AMD Linux-x86-64 platform. A description of this platform, its configuration, and its results is available in Appendix C.2.

4.1.2 System and Runtime Software. Both testbeds run Debian 11 with Linux V6.1.27 as the base kernel. The base JVM is the open-source HotSpot JVM from OpenJDK 24 (build `jdk-24+2`) [17]. Our evaluation uses the server optimized build of HotSpot, which we compiled with -O3 using the GNU Compiler Collection (GCC) 10.2.1. All of our experiments enable ZGC with `-XX:+UseZGC` and log collection decisions to a file on disk using HotSpot's built-in logging facilities. The DaCapo benchmarks always invoke `System.gc()` between iterations. To evaluate OppZGC without influence from explicit collections, our experiments also disable explicit collections with `-XX:+DisableExplicitGC`.

4.1.3 Common Experimental Configuration. Each experiment runs the benchmark on an otherwise idle machine, and all results report the mean of five experimental runs. To estimate and report variability in our results, we compute 95% confidence intervals (CIs) for the difference between the means of the experimental and default configurations, using the unequal-variance (Welch) formulation with Welch–Satterthwaite degrees of freedom, as described in Georges et al. [9]. Figures that report results for individual benchmarks plot these intervals as error bars around the sample means.[3] Some figures report average results over

[3]Figures 2 and 3 omit these intervals to avoid cluttering the graphs.

a set of benchmarks. Average results relative to a baseline configuration present the geometric mean and non-relative averages present the arithmetic mean.

4.2 Benchmarks

We evaluate OppZGC with a selection of benchmarks from the DaCapo Chopin suite (v. 23.11-MR2) [5] and the compute- and memory-intensive SPECjbb 2015 (v. 1.03) [20].

4.2.1 DaCapo Chopin. DaCapo Chopin is a widely used Java benchmark suite consisting of real and open-source Java applications with a diverse range of computing behaviors, goals, and resource needs. To focus our evaluation, we omit benchmarks that show no measurable steady-state performance difference with default input when the number of computing cores available to the application is reduced from 16 to 2.[4] We also omit tradebeans and tradesoap, which are not supported by our JDK (OpenJDK version 24) [8]. Table 4 in Appendix B lists our selected DaCapo benchmarks along with performance and memory characteristics for the baseline ZGC configurations included in this study.

DaCapo includes multiple input sizes for most applications. Our study uses the default input and large input (if one is included) for each selected benchmark, for a total of 22 inputs over 12 applications. The DaCapo suite also includes a harness program that runs the benchmark for a given number of iterations or until convergence, which occurs when the standard deviation of the last three iterations divided by their average execution time is less than 3%. Since concurrent GC can potentially affect execution time of individual iterations (and thus convergence), we opted to set the number of iterations manually for each benchmark-input pair. For the default inputs, we use a number of iterations that executes the workload for 30 seconds to 150 seconds in the baseline configuration. All the large inputs were run with exactly 5 iterations per run. [5]

For each benchmark, we report both *startup time*, which is the execution time of the first iteration, and *steady-state time*, which we compute as the average execution time of every iteration aside from the first two (i.e., the startup iteration plus one warmup iteration). In this way, our steady-state measurements avoid the period with heaviest JIT compilation while still capturing GC costs over a common number of iterations. Ten of the benchmark-input pairs, marked in Tables 3 and 4 with †, also report request-based latencies as their primary performance metric. For these workloads, we report the simple (i.e., non-metered) p50, p90, p99, p99.9, and p99.99 latencies over the steady-state iterations.

[4]Specifically, we use `numactl` to limit the CPU set available to the JVM process and set `-XX:ActiveProcessorCount` to the appropriate core count.
[5]Backtesting against the DaCapo convergence formula shows that the number of iterations we used is sufficient for all but five benchmarks to converge with baseline ZGC: *jython-default*, *pmd-large*, *sunflow-default*, *sunflow-large*, and *xalan-large*.

4.2.2 SPECjbb2015. SPECjbb is a popular Java performance analysis benchmark for server platforms that scales with both compute and memory resources. It includes options for running with multiple JVMs or across multiple hosts, but for this work, we run it using a single JVM in composite mode on our single node platform.

The workload models a supermarket IT infrastructure, including Point of Sale (POS) transactions, receipt and inventory management, interactions with suppliers, and data analytics. The SPECjbb driver injects transactions at a specific rate called the Injection Rate (IR), and the system under test completes the transactions at a measured rate known as jOPS. The full benchmark consists of five phases: High Bound Injection Rate (HBIR) Search, Response-Throughput (RT) curve building, Validation, Profiling, and Reporting. Performance is determined by the first two phases. The HBIR phase quickly estimates throughput capabilities of the system under test, and the RT curve building then walks the injection rate up more finely, locating the point where throughput stops scaling with injection rate, known as saturation.

The benchmark reports two primary performance measurements: the maximum jOPS, or throughput at the saturation point, and critical jOPS, which measures throughput with some acceptable latency under a range of service level agreements (SLAs). Specifically, SPECjbb defines five SLAs at 10, 25, 50, 75, and 100 ms. As the benchmark sweeps the injection rate, it finds the approximate throughput level at which the p99 response time crosses the SLA. Critical jOPS is the geometric mean of those five throughput values.

5 Evaluation

5.1 Selecting the Growth Ratio and Period

As discussed in Section 3, OppZGC has two parameters that affect its scheduling decisions: the growth ratio α and the period δ. To select parameter values for this study, we conducted a sweep of different α values and δ values with the DaCapo benchmarks with their default inputs. Specifically, we tested α values of 1.1, 1.0, 0.9, 0.8, 0.5, and 0.1 with δ values of 10 ms, 20 ms, 40 ms, 80 ms, and 160 ms, for a total of 30 different configurations. Of these, the three configurations that achieved the lowest maximum heap size without degrading the average steady-state performance: (α=1.0, δ = 10 ms), (α=1.0, δ = 20 ms), and (α=0.9, δ = 20 ms). We then ran these three configurations with our full benchmark set. This section shows detailed results for OppZGC with α=1.0, δ=20 ms, which achieves the best average performance of the three configurations, and α=0.9, δ=20 ms, which achieves the best average memory savings.

5.2 OppZGC Collection Statistics

Table 3 presents collection statistics for baseline ZGC and two OppZGC configurations across our complete benchmark set. As expected, both OppZGC configurations collect

Benchmark	Input	Baseline ZGC				OppZGC, $\alpha = 1.0$, $\delta = 20$ ms						OppZGC, $\alpha = 0.9$, $\delta = 20$ ms					
		Minor	Major	MB/s	GCt (s)	Minor		Major		MB/s	GCt (s)	Minor		Major		MB/s	GCt (s)
		#	#			#	%	#	%			#	%	#	%		
cassandra†	Default	1	6	5.6	3.9	45	100	8	61	21.1	14.0	55	100	9	78	27.5	16.4
fop		1	4	11.6	10.4	39	100	9	70	40.0	28.8	54	100	12	92	46.5	33.5
graphchi		2	5	1.4	0.6	32	100	6	45	12.6	1.6	62	100	8	68	25.2	2.6
h2†		35	6	42.3	50.5	52	53	13	73	40.4	53.2	55	65	9	67	50.5	54.2
jython		2	4	6.0	8.0	70	100	7	86	16.3	19.6	92	100	10	90	15.8	20.7
lusearch†		1	19	0.7	1.6	18	100	25	14	1.0	2.1	20	100	26	11	1.2	2.0
pmd		7	5	24.8	4.7	42	100	8	60	90.6	15.9	56	100	10	58	110.2	22.2
spring†		11	10	1.6	3.7	128	100	13	49	3.1	8.3	942	100	18	83	3.5	27.9
sunflow		13	20	3.9	2.0	29	88	24	14	5.7	2.4	33	98	26	16	5.3	2.4
tomcat†		0	10	0.8	2.9	217	100	12	92	1.3	7.5	587	100	24	97	1.6	15.2
xalan		9	9	2.2	0.8	38	100	13	41	5.6	1.4	53	100	13	49	19.0	1.5
zxing		2	4	65.8	1.7	33	100	10	56	94.9	2.9	43	100	11	55	98.9	3.2
cassandra†	Large	13	8	36.2	31.5	74	100	16	29	61.4	67.3	107	100	21	35	91.0	81.1
graphchi		0	88	4.7	13.2	42	100	77	4	12.1	14.5	134	100	33	14	23.0	16.2
h2†		552	33	152.2	2922.7	573	5	38	18	149.8	2841.3	572	5	40	19	150.8	2850.4
jython		38	7	23.6	105.7	213	100	28	38	24.7	160.0	622	100	74	8	25.6	264.2
lusearch†		0	20	0.2	40.9	4	100	108	1	0.2	41.6	10	100	210	1	0.2	42.0
pmd		26	9	612.4	233.3	37	33	11	40	620.1	227.3	38	34	13	45	616.1	234.3
spring†		32	26	0.9	8.7	160	100	24	13	1.5	12.0	2818	100	30	89	1.7	65.8
sunflow		11	51	3.9	3.9	26	80	58	9	4.6	4.5	36	98	60	7	4.5	4.7
tomcat†		0	31	0.5	6.3	326	100	10	92	0.5	10.1	1421	100	26	96	0.5	30.5
xalan		11	18	1.9	1.4	45	100	14	14	3.2	1.5	49	100	14	16	3.7	1.5
SPECjbb2015	Single JVM	703	35	100	1684.2	744	25	37	15	105	1781.3	919	54	43	15	125	2314.9

Table 3. Baseline ZGC and OppZGC collection statistics. For each ZGC configuration, the columns show the number of minor and major collections, (for OppZGC only) the percentage of minor and major collections caused by the opportunistic rules, the data copy rate (i.e., MB copied for ZPage compaction or object promotions per second), and the aggregate CPU time of all ZGC workers in seconds. All configurations shown here use a maximum heap size of 32 GB.

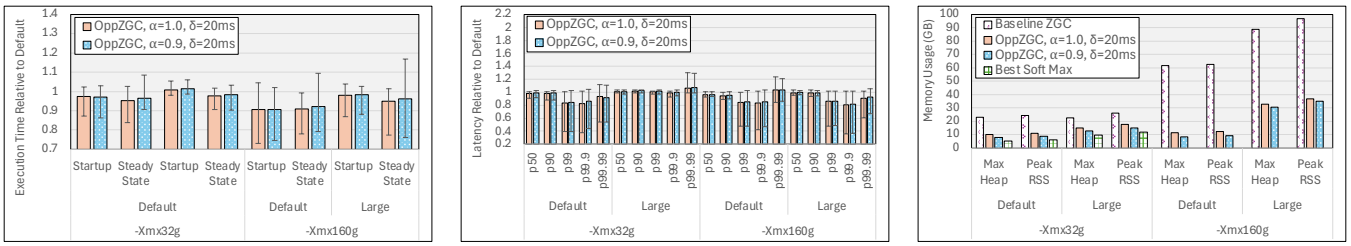
garbage more aggressively than baseline ZGC. Most of these benchmarks leave at least some of the 16 available computing cores idle or underutilized during their execution. To take advantage of the available CPU capacity, OppZGC runs from several dozen up to a few thousand additional collection cycles compared to baseline ZGC.

Increasing the number of collections also leads to higher data copy rates and requires additional CPU time to complete the extra collection work. On average, OppZGC increases the CPU time spent on GC by 1.7× with $\alpha = 1.0$ and by 2.3× with $\alpha = 0.9$, compared to the baseline ZGC configuration. However, the impact of OppZGC on GC effort can vary widely for different benchmarks. For example, *spring* exhibits GC CPU time increases ranging from 1.4× to 7.6× with OppZGC. Other benchmarks, such as *h2* and *lusearch*, see little or no increase in GC activity with OppZGC. For these benchmarks, the mutators exhaust CPU capacity throughout most of their execution, and thus, OppZGC forgoes additional collection cycles, thereby avoiding performance losses. Notably, this extra collection work does not increase energy costs. On average, package power is essentially unchanged (< 1% increase), and package energy is reduced slightly (by < 2%, matching performance improvements) for the DaCapo benchmarks with OppZGC using either $\alpha = 1.0$ or $\alpha = 0.9$ and -Xmx32g.

5.3 Evaluation with 16 Cores

5.3.1 DaCapo Chopin Performance. Figures 5a and 5b present the average performance of OppZGC in terms of startup and steady-state execution time, as well as the p50, p90, p99, p99.9, and p99.99 latencies. Both panels show results for the two OppZGC configurations relative to baseline ZGC performance for the same input and -Xmx value. Error bars show the best (low end) and worst (high end) performing benchmark for each configuration. Detailed per-benchmark results are also presented in Appendix C.1.3.

Our results show that both OppZGC configurations achieve similar startup and steady-state performance to baseline ZGC in most cases, with only a few negative exceptions. Even when the CPU is underutilized, increasing the rate of collection may still slow mutator progress because doing so increases the frequency of barrier slow paths and can evict warm data from processor caches. However, we find that mutator interference is somewhat limited for both OppZGC configurations. In some cases, startup time appears to degrade by 10% or more, but run-to-run variance for startup time is relatively high and these differences do not fall outside the 95% CIs. In the worst case, steady-state execution time increases by about 6% for *xalan-large* with -Xmx32g.



(a) Startup and steady-state time. **(b)** Latency distribution. **(c)** Maximum heap used and peak RSS.

Figure 5. OppZGC performance and memory usage, averaged over the DaCapo benchmark groups. (a) and (b) show results relative to baseline ZGC and include error bars showing the best and worst result from each group (lower is better). Results in (c) are in GB. The -Xmx32g group also shows results the best per-benchmark soft max configuration.

On the other hand, several benchmarks exhibit more substantial startup and steady-state performance improvements compared to baseline ZGC. In the best cases, individual benchmarks can improve by more than 20% in startup (*xalan-default*) or steady-state (*pmd-default*) performance. Similar to the soft max results in Section 2.2, we find that these performance improvements are primarily driven by reduced paging overhead: fewer page faults enable better startup performance, and a smaller heap footprint improves TLB efficiency. For the default inputs with a 160 GB heap limit, these effects improve the startup and steady-state performance by about 9%, on average.

Request latencies from p50 up through p99.9 are essentially unharmed by either OppZGC configuration. However, we find that OppZGC can degrade the p99.99 tail latency, in a few cases. With -Xmx32g, both OppZGC configurations increase p99.99 latency for *cassandra-large* by about 30%, and with the larger heap maximum, OppZGC increases p99.99 latencies for *h2-default* by between 16% and 24% and for *lusearch-default* by 15% to 21%. Despite these few exceptions, OppZGC achieves typical and tail latencies very similar to baseline ZGC, and for the vast majority of workloads and execution scenarios that we tested, it preserves the low-latency behavior that ZGC aims to provide.

5.3.2 DaCapo Chopin Memory. Figure 5c presents the average memory usage of baseline ZGC and the two OppZGC configurations in terms of maximum heap used and peak RSS, both in GB. Results are grouped by benchmark input and maximum heap size. For comparison, we plot an additional bar in the -Xmx32g results based on our experiments varying the soft max parameter in Section 2.2. Specifically, this bar shows the average memory usage of the per-benchmark soft max value that exhibits the lowest heap usage without hurting any performance metric (i.e., startup, steady-state, or latency) relative to baseline ZGC.

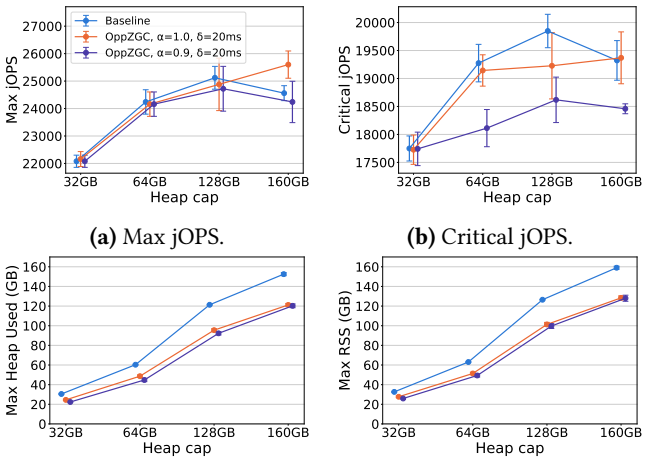
OppZGC substantially reduces memory usage for at least 10 (of 12) default input benchmarks (depending on α and the hard heap limit) and for at least 6 (of 10) of the large input benchmarks compared to baseline ZGC. With -Xmx32g, OppZGC reduces maximum heap usage by 61%, on average, with $\alpha = 1.0$ and by 75% with $\alpha = 0.9$; with -Xmx160g, the reductions are 86% and 90%, respectively.[6] As expected, the benchmarks where OppZGC is least effective at reducing memory usage are the same benchmarks with highest CPU utilization in the baseline configuration. For these benchmarks, OppZGC exhibits similar memory usage to baseline ZGC because it still checks the same scheduling rules as baseline ZGC, even if the opportunistic rules never fire.

The results allow two other notable observations. First, reducing heap usage produces similar savings in resident set size. Although other factors, such as stack memory and the distribution of objects on ZPages, can increase peak RSS for some workloads, the impact of these factors is relatively small for these benchmarks. Additionally, while OppZGC substantially reduces memory usage compared to baseline ZGC, a well-tuned soft maximum can still achieve better memory savings for 13 of our 22 benchmarks. On average, the best soft max configuration uses 2.8 GB less heap memory than OppZGC with $\alpha = 0.9$ (with -Xmx32g). However, these additional memory savings come at the cost of offline, per-benchmark tuning, which is impractical for many applications. OppZGC achieves these improvements using the same configuration for the entire benchmark set, and does not require customization or tuning for each workload.

5.3.3 SPECjbb2015. Since SPECjbb performance typically scales with the availability of memory resources, we evaluated OppZGC with SPECjbb with four different maximum heap sizes: 32 GB, 64 GB, 128 GB, and 160 GB. Figure 6 presents the max jOPS (maximum throughput), critical jOPS (throughput achieved given some latency constraints), maximum heap usage, and peak RSS for the baseline and two OppZGC configurations (with $\alpha = 1.0$ and $\alpha = 0.9$) with each maximum heap size.

As with DaCapo, the results show that both OppZGC configurations have little to no impact on peak throughput,

[6] These percentages are computed for each benchmark as the reduction relative to baseline ZGC, then averaged over the full group. Since 5c presents the mean heap occupancy of each configuration as a scalar, its implied ratios show a similar trend, but do not match these percentages exactly.



(a) Max jOPS. **(b)** Critical jOPS.

(c) Max heap used (GB). **(d)** Peak RSS (GB).

Figure 6. SPECjbb performance and memory usage with OppZGC and baseline ZGC. Each line shows results with different -Xmx limits: 32g, 64g, 128g, and 160g.

regardless of the maximum heap size. However, the more aggressive OppZGC configuration can increase request latency in some cases, reducing the critical jOPS. In the worst case, OppZGC with $\alpha = 0.9$ and a 128 GB maximum heap size reduces critical jOPS by 6.2% compared to baseline ZGC. The more conservative $\alpha = 1.0$ setting eliminates this degradation and achieves essentially the same latency-constrained throughput as baseline ZGC.

While OppZGC has minimal impact on SPECjbb performance, it still reduces memory usage substantially compared to baseline ZGC. In this case, there is also very little difference in memory usage between the two OppZGC configurations, with $\alpha = 0.9$ using slightly less memory at most maximum heap sizes. For $\alpha = 1.0$, reductions in maximum heap occupancy range from 19.4% with -Xmx64g to 21.3% with -Xmx128g, which corresponds to reductions in peak RSS ranging from 15% to 20%. While these reductions are not as pronounced as the average savings with DaCapo, they demonstrate that OppZGC can even reduce memory usage for CPU-intensive workloads with efficient scaling.

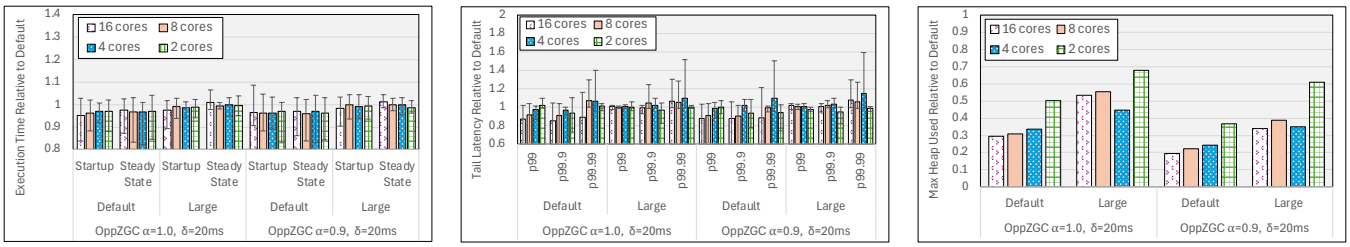
5.4 Evaluation with Varying CPU Capacity

5.4.1 Performance with Varying CPU Capacity. To evaluate OppZGC with varying CPU capacity, we conducted a series of experiments with the Java process restricted to use only 8, 4, and 2 of the computing cores on our Intel-based platform. Figures 7a and 7b show the average performance in terms of startup time, steady-state time, and tail latencies for each core restriction. The panels show average results, grouped by input size, for the two OppZGC configurations with different core restrictions relative to a baseline ZGC configuration *with the same core restriction*. All results use a maximum heap size of 32 GB. Similar to Figures 5a and 5b, error bars show the best (low end) and worst (high end) performing benchmark for each configuration. For added context, each panel also reprints the results for the 16-core configuration from Section 5.3. Additionally, since typical latencies (i.e., p50 and p90) are essentially unaffected by OppZGC, Figure 7b omits these results and plots only the p99, p99.9, and p99.99 tail latencies to reduce clutter in the graph. Full results are available in Appendix C.1.3.

In general, reducing the number of computing cores available does not change the relative startup or steady-state performance of OppZGC. On average, each OppZGC configuration exhibits startup and steady-state performance that is about the same or better than baseline ZGC with the same number of cores. In contrast, there are a few cases where reducing the available CPU capacity causes additional degradations to the tail latency, especially at p99.99. In the worst cases, OppZGC increases p99.99 latency for *cassandra* by 40% to 57%, depending on the benchmark input and α setting, with execution restricted to only four computing cores.

Notably, the most pronounced degradations all occur when the application is limited to 4 computing cores, rather than 8 or 2. With only two cores available, both cores are almost always saturated with mutator activity for many of the benchmarks (which prevents the opportunistic rules from firing), while with 16 and 8 cores available, at least some CPU is left unused for most of the run for most benchmarks, which allows for additional collection with minimal cost. However, with only four cores available, several of our benchmarks, and especially *cassandra*, oscillate between periods where the CPUs are underutilized and then quickly become entirely saturated. Thus, there are more cases where the OppZGC rules incorrectly predict that there is sufficient CPU capacity to support a collection cycle. For these cases, a more conservative OppZGC configuration with a longer period (δ) can help mitigate these performance losses. For *cassandra* with four cores, we found that lengthening the period to a much more conservative 10 s eliminates most tail latency degradations (though the large input still exhibits a 25% increase in p99.99 latency), while still reducing maximum heap usage by 57% and 76% for the default and large inputs.

5.4.2 Memory Usage with Varying CPU Capacity. Finally, Figure 7c presents the maximum heap occupancy for the two OppZGC configurations relative to baseline ZGC for each core configuration, averaged over each benchmark-input set. We find that limiting the workload to only 8 or 4 cores has only a minor impact on the average memory savings. In these configurations, OppZGC still frequently predicts (sometimes incorrectly, as discussed above) there is enough CPU capacity to support additional collection cycles for most workloads. However, with only two cores for the entire process, OppZGC becomes more conservative and more often forgoes collections to allow the mutators to operate unimpeded. Overall, when execution is restricted to only two cores, OppZGC with $\alpha = 0.9$ reduces the maximum heap



(a) Startup and steady-state time. **(b)** Tail latencies. **(c)** Maximum heap occupancy.

Figure 7. OppZGC with varying CPU capacity, relative to baseline ZGC at the same capacity, averaged over the DaCapo benchmark groups. Error bars in (a) and (b) show the best and worst result from each group (lower is better).

occupancy for 7 of 12 default input benchmarks and for 3 of 10 large input benchmarks, resulting in average reductions in heap occupancy of 63% and 39% for the two input sets, respectively. In contrast, the same OppZGC configuration with 16-core execution reduces heap memory usage by 80% and 65% for each input set.

6 Related Work

Since the introduction of tracing garbage collectors, which defer reclamation of dead objects rather than freeing them immediately [16], researchers have sought to understand how to balance memory usage with collection costs. Several works have studied and quantified the computational costs of different types of collectors, including generational collectors [2, 6, 11, 14]. Bacon et al. proposed a unified cost model for trace-based and reference-counting collectors and studied tradeoffs across different designs [3]. Wilson [24] surveyed the canonical collection algorithms and described their tradeoffs, including Baker's algorithm [4], which is the basis of ZGC's load barriers. Yang and Wrigstad also provided a comprehensive description of an earlier version of ZGC (from JDK 15) [25].

Several prior works have observed that effective collection can reduce paging costs and have designed collection strategies to exploit this consequence [1, 7, 10, 26, 27]. Kolokasis et al. [12] balance GC and I/O cache allocation, partitioning a fixed DRAM budget between the heap and the page cache using the CPU time lost to GC and to I/O stalls. White et al. [23] employ a principled mathematical approach, tuning a PID controller to track a heap size more responsively than the heuristic-based mechanisms of Jikes and HotSpot. In contrast to OppZGC, each of these works sets the heap size directly as a means to control GC cost. Moreover, these works target collectors that serialize reclamation with the mutators, where the costs are paid in pause frequency and duration. Since concurrent collectors do most of their work as the mutators run, contention for the CPU can worsen mutator performance. OppZGC aims to avoid this contention by only running extra collections in otherwise idle CPU capacity.

Perhaps most closely related to this study are prior works that use system load to influence collector scheduling. Zhao et al. [29] schedule concurrent work into troughs in system load to protect latency-critical tenants co-located with best-effort jobs. Shimchenko et al. [19] schedule in troughs while also building priority-inversion-aware slack scheduling directly into the JVM. Both of these works use the same signal of system load as OppZGC, but they use this information for scheduling collections to reduce latency instead of increasing collection frequency to restrict memory usage.

Another important related effort, which has since been adopted by a JEP for a future OpenJDK release [31], schedules collections by targeting a ratio of collector to mutator CPU time and letting heap size follow [22]. This ratio, however, is independent of CPU capacity. It aims for the same target whether the CPU is saturated or idle, though the marginal cost of collection is contention for CPU. The policy is thus vulnerable to scenarios where the available CPU capacity shifts during execution and becomes misaligned with the chosen target ratio. In contrast, OppZGC triggers extra collections only when sufficient idle CPU capacity is detected.

7 Conclusion

This work presents Opportunistic ZGC: a feedback-directed scheduling policy for concurrent collectors that exploits otherwise unused CPU capacity to constrain the heap, while keeping the costs of extra collection cycles low. While the approach includes two tunable parameters, this study finds parameter values that reduce memory usage for a diverse set of benchmarks. Thus, this work fills a gap between current practices of relatively conservative GC scheduling, which permits the heap to expand to use the space allowed by a predefined maximum, and per-application tuning, which allows users to constrain the heap with knobs for setting a soft heap maximum or a target ratio for collector-to-mutator CPU time. On our 16-core Intel platform, OppZGC reduces maximum heap usage for our DaCapo benchmarks by between 61% and 90%, on average, compared to default ZGC, depending on configuration. For SPECjbb, OppZGC reduces memory usage by up to 21% in large memory configurations, with no cost to maximum or latency-constrained throughput.

In scenarios with very limited CPU capacity, OppZGC automatically adjusts GC scheduling to avoid stealing scarce

CPU from mutator threads. In some configurations, bursty workloads can cause OppZGC to mispredict the CPU capacity available for collection, which can hurt request latency performance in the extreme tail. Future efforts will focus on extending OppZGC to identify scenarios where opportunistic collection cycles can hurt performance and designing strategies to forgo these extra collections or mitigate their costs. However, for the vast majority of applications and execution scenarios, this work shows that OppZGC effectively constrains the application heap, with minimal harm to startup time, throughput, and request latencies.

References

[1] Rafael Alonso and Andrew W. Appel. 1990. An Advisor for Flexible Working Sets. In *Proceedings of the 1990 ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems (SIGMETRICS '90)*. Association for Computing Machinery, New York, NY, USA, 153–162. doi:10.1145/98457.98753

[2] Andrew W. Appel. 1989. Simple Generational Garbage Collection and Fast Allocation. *Software: Practice and Experience* 19, 2 (1989), 171–183. doi:10.1002/spe.4380190206

[3] David F. Bacon, Perry Cheng, and V. T. Rajan. 2004. A Unified Theory of Garbage Collection. In *Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*. ACM, 50–68. doi:10.1145/1028976.1028982

[4] Henry G. Baker. 1978. List Processing in Real Time on a Serial Computer. *Commun. ACM* 21, 4 (April 1978), 280–294. doi:10.1145/359460.359470

[5] Stephen M. Blackburn, Zixian Cai, Rui Chen, Xi Yang, John Zhang, and John N. Zigman. 2025. Rethinking Java Performance Analysis. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 940–954. doi:10.1145/3669940.3707217

[6] Stephen M. Blackburn, Perry Cheng, and Kathryn S. McKinley. 2004. Myths and Realities: The Performance Impact of Garbage Collection. In *ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems (ACM SIGMETRICS Performance Evaluation Review 32(1))*. ACM Press, 25–36. doi:10.1145/1005686.1005693

[7] Rodrigo Bruno, Paulo Ferreira, Ruslan Synytsky, Tetiana Fydorenchyk, Jia Rao, Hang Huang, and Song Wu. 2018. Dynamic Vertical Memory Scalability for OpenJDK Cloud Applications. In *Proceedings of the 2018 ACM SIGPLAN International Symposium on Memory Management (ISMM 2018)*. Association for Computing Machinery, New York, NY, USA, 59–70. doi:10.1145/3210563.3210567

[8] DaCapo Benchmark Suite. 2025. tradebeans and tradesoap fail on JDK 24. DaCapo benchmark suite issue #350; WildFly 26, used by the Trade* workloads, is incompatible with Java releases newer than 21. https://github.com/dacapobench/dacapobench/issues/350

[9] Andy Georges, Dries Buytaert, and Lieven Eeckhout. 2007. Statistically Rigorous Java Performance Evaluation. In *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA '07)*. Association for Computing Machinery, New York, NY, USA, 57–76. doi:10.1145/1297027.1297033

[10] Chris Grzegorczyk, Sunil Soman, Chandra Krintz, and Rich Wolski. 2007. Isla Vista Heap Sizing: Using Feedback to Avoid Paging. In *Proceedings of the International Symposium on Code Generation and Optimization (CGO '07)*. IEEE Computer Society, 325–340. doi:10.1109/CGO.2007.20

[11] Matthew Hertz and Emery D. Berger. 2005. Quantifying the Performance of Garbage Collection vs. Explicit Memory Management. In *Proceedings of the 20th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA '05)*. Association for Computing Machinery, New York, NY, USA, 313–326. doi:10.1145/1094811.1094836

[12] Iacovos G. Kolokasis, Shoaib Akram, Foivos S. Zakkak, Polyvios Pratikakis, and Angelos Bilas. 2026. FlexHeap: Dynamic I/O-Aware Heap Resizing for Managed Applications. *Proceedings of the ACM on Programming Languages* 10, PLDI (June 2026), 5–28. doi:10.1145/3808247

[13] Per Lidén and Stefan Karlsson. 2018. *JEP 333: ZGC: A Scalable Low-Latency Garbage Collector (Experimental)*. Technical Report. OpenJDK. http://openjdk.java.net/jeps/333

[14] Henry Lieberman and Carl Hewitt. 1983. A Real-Time Garbage Collector Based on the Lifetimes of Objects. *Commun. ACM* 26, 6 (June 1983), 419–429. doi:10.1145/358141.358147

[15] Linux man-pages project. 2024. getrusage(2) — Linux manual page. https://man7.org/linux/man-pages/man2/getrusage.2.html

[16] John McCarthy. 1960. Recursive Functions of Symbolic Expressions and Their Computation by Machine, Part I. *Commun. ACM* 3, 4 (April 1960), 184–195. doi:10.1145/367177.367199

[17] OpenJDK Contributors. 2024. OpenJDK JDK mainline, tag jdk-24+2, commit 50bed6c. Commit 50bed6c67b1edd7736bdf79308d135a4e1047ff0. https://github.com/openjdk/jdk/commit/50bed6c67b1edd7736bdf79308d135a4e1047ff0

[18] Oracle Corporation. 2025. The Z Garbage Collector. Published: HotSpot Virtual Machine Garbage Collection Tuning Guide, Java SE 24. https://docs.oracle.com/en/java/javase/24/gctuning/z-garbage-collector.html

[19] Marina Shimchenko, Erik Österlund, and Tobias Wrigstad. 2025. Monk: Opportunistic Scheduling to Delay Horizontal Scaling. *The Art, Science, and Engineering of Programming* 10, 1 (Feb. 2025), 1. arXiv:2502.20522 [cs.PL]. doi:10.22152/programming-journal.org/2026/10/1

[20] Standard Performance Evaluation Corporation. 2015. SPECjbb2015 Benchmark, version 1.03. Standard Performance Evaluation Corporation. https://www.spec.org/jbb2015/

[21] Stefan Karlsson. 2021. *JEP 439: Generational ZGC*. Technical Report. https://openjdk.org/jeps/439

[22] Sanaz Tavakolisomeh, Marina Shimchenko, Erik Österlund, Rodrigo Bruno, Paulo Ferreira, and Tobias Wrigstad. 2023. Heap Size Adjustment with CPU Control. In *Proceedings of the 20th ACM SIGPLAN International Conference on Managed Programming Languages and Runtimes (MPLR 2023)*. Association for Computing Machinery, New York, NY, USA, 114–128. doi:10.1145/3617651.3622988

[23] David R. White, Jeremy Singer, Jonathan M. Aitken, and Richard E. Jones. 2013. Control Theory for Principled Heap Sizing. In *Proceedings of the 2013 International Symposium on Memory Management*. ACM, Seattle Washington USA, 27–38. doi:10.1145/2464157.2466481

[24] Paul R. Wilson. 1992. Uniprocessor Garbage Collection Techniques. In *Proceedings of the International Workshop on Memory Management (IWMM '92) (Lecture Notes in Computer Science, Vol. 637)*. Springer-Verlag, Berlin, Heidelberg, 1–42. doi:10.1007/BFb0017182

[25] Albert Mingkun Yang and Tobias Wrigstad. 2022. Deep Dive into ZGC: A Modern Garbage Collector in OpenJDK. *ACM Transactions on Programming Languages and Systems* 44, 4 (Dec. 2022), 1–34. doi:10.1145/3538532

[26] Ting Yang, Emery D. Berger, Scott F. Kaplan, and J. Eliot B. Moss. 2004. Automatic Heap Sizing: Taking Real Memory into Account. In *Proceedings of the 4th International Symposium on Memory Management (ISMM '04)*. Association for Computing Machinery, New York, NY, USA, 61–72. doi:10.1145/1029873.1029881

[27] Ting Yang, Emery D. Berger, Scott F. Kaplan, and J. Eliot B. Moss. 2006. CRAMM: Virtual Memory Support for Garbage-Collected Applications. In *7th USENIX Symposium on Operating Systems Design and Implementation (OSDI 06)*. USENIX Association, Seattle, WA. https://www.usenix.org/conference/osdi-06/cramm-virtual-memory-support-garbage-collected-applications

[28] Taiichi Yuasa. 1990. Real-Time Garbage Collection on General-Purpose Machines. *Journal of Systems and Software* 11, 3 (1990), 181–198. doi:10.1016/0164-1212(90)90084-Y

[29] Junxian Zhao, Aidi Pi, Xiaobo Zhou, Sang-Yoon Chang, and Chengzhong Xu. 2022. Improving Concurrent GC for Latency Critical Services in Multi-tenant Systems. In *Proceedings of the 23rd ACM/IFIP International Middleware Conference (Middleware '22)*. Association for Computing Machinery, New York, NY, USA, 43–55. doi:10.1145/3528535.3531515

[30] Erik Österlund. 2024. ZGC Automatic Heap Sizing. Presented at the JVM Language Summit (JVMLS). https://inside.java/2024/11/09/jvmls-zgc/

[31] Erik Österlund. 2026. *JEP Draft 8377305: Automatic Heap Sizing for ZGC*. Technical Report. Published: OpenJDK. https://openjdk.org/jeps/8377305

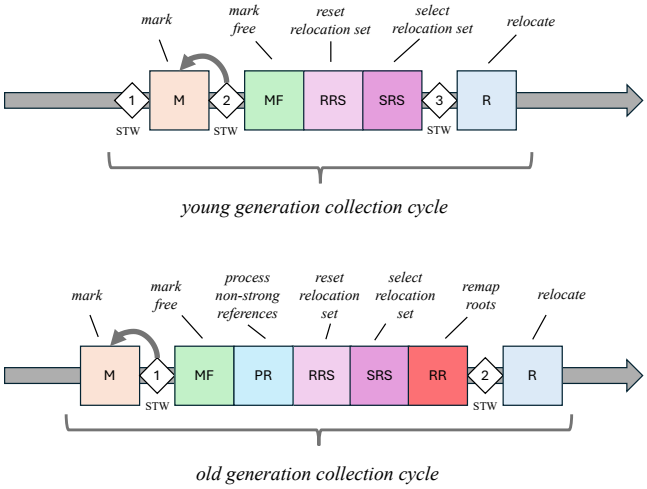


Figure 8. Young and old collection cycles in ZGC. Old collection does not require an initial pause because it always runs as part of a major GC, immediately after young collection. Box widths are not proportional to phase lengths.

A Young and Old Collection Cycles for ZGC

Figure 8 illustrates the young and old collection cycles in ZGC. Young generation collection consists of three brief stop-the-world (STW) pauses and five concurrent phases: mark (M), mark free (MF), reset relocation set (RRS), select relocation set (SRS), and relocate (R). The initial pause flips the global status bits so that the barrier instructions in the mutators see that a new marking phase has begun and also scans the mutators' stacks and other roots to seed the mark. Next, the mark phase traverses the young object graph from the roots, marking any objects it reaches. The second pause attempts to terminate the mark, but a concurrent mutator might have created additional work during marking (e.g., by writing an object reference) that is not yet complete. In that case, the cycle retries the mark phase until it completes with no additional work. When marking is complete, the mark free phase releases some per-worker marking data structures. Next, the cycle prepares for object relocation by: 1) clearing the previous cycle's relocation structures, and 2) scanning the candidate ZPages and selecting the sparse ZPages from which it will evacuate live objects. The third pause sets the global status bits to indicate the beginning of the relocation phase and relocates root objects so that mutators immediately see the forwarded root references. The relocation phase then proceeds concurrently with the mutators, moving the live objects from the selected ZPages to new ZPages and freeing the sparse (now unused) ZPages. Note that, during this process, if a mutator attempts to use an object that is marked for relocation but has not yet been moved, the mutator itself will move the object via the load barrier.

The old collection cycle largely mirrors the young cycle, but with one fewer pause and two additional phases. Since each old collection is always immediately preceded by a young collection, the synchronization steps that are necessary prior to the mark phase, including updating the global status bits, can be completed during the young cycle's initial pause. Hence, the old cycle omits the initial STW pause. The extra phases in the old cycle are process non-strong references (PR) and remap roots (RR). Java has several types of "non-strong" references (e.g., soft and weak references) whose referents are reclaimed when they are no longer strongly reachable across the entire heap. The PR phase resolves these non-strong references using the strong reachability information established during marking. Finally, the remap roots phase heals stale pointers left by previous cycles' relocations before the global relocation bits flip again. In this way, mutators avoid misinterpreting pointers that are stale across multiple relocation cycles.

B DaCapo Benchmark Characteristics

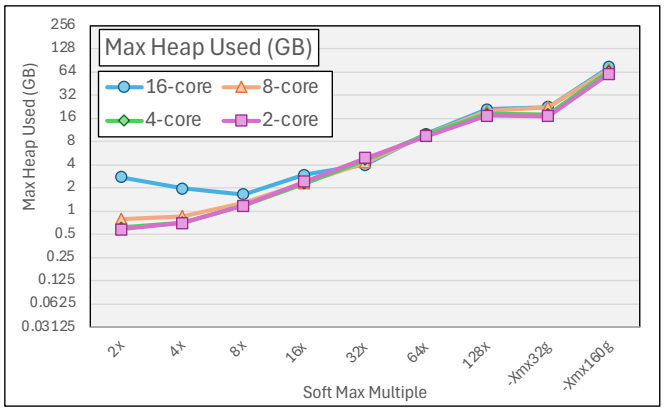
Benchmark	Iters	Min MB	Baseline ZGC with -Xmx32g			Baseline ZGC with -Xmx160g		
			Time (s)	CPU %	Max GB	Time (s)	CPU %	Max GB
Default Input								
cassandra†	10	182	67.6	188.6	25.6	67.7	189.6	29.2
fop	100	26	60.8	199.2	17.6	62.4	187.0	21.6
graphchi	10	196	54.7	312.2	25.6	58.2	331.8	48.0
h2†	10	980	37.5	452.2	18.1	42.4	418.4	108.4
jython	10	58	64.3	154.8	15.1	71.0	140.6	27.1
lusearch†	10	42	129.2	1,431.8	24.1	134.5	1,422.8	48.0
pmd	30	216	45.6	628.4	24.0	58.0	653.0	104.7
spring†	30	98	104.8	1,209.2	25.6	113.3	1,153.6	143.3
sunflow	30	50	118.6	1,501.0	25.7	106.9	1,441.6	60.2
tomcat†	20	46	149.0	318.4	20.3	149.4	314.2	48.0
xalan	50	40	42.1	1,363.6	25.7	49.7	1,260.0	65.4
zxing	50	98	78.4	1,265.6	25.7	85.0	1,278.8	34.5
Large Input								
cassandra†	5	270	116.3	471.8	20.9	126.5	456.0	115.7
graphchi	5	1,166	1,041.1	493.0	18.0	1,036.2	493.0	48.0
h2†	5	14,230	710.3	727.4	26.2	692.9	688.0	115.5
jython	5	60	620.3	123.2	23.3	660.1	119.4	117.0
lusearch†	5	118	2,685.9	1,577.6	7.7	2,749.7	1,574.0	48.0
pmd	5	4,544	76.5	897.2	28.5	76.5	798.8	102.2
spring†	5	134	306.0	1,298.4	25.6	313.1	1,258.0	152.0
sunflow	5	258	267.7	1,550.8	25.7	262.4	1,533.0	58.4
tomcat†	5	56	354.3	293.0	24.3	355.1	292.4	48.0
xalan	5	52	67.2	1,342.4	25.7	73.0	1,248.6	85.9

Table 4. DaCapo benchmarks with baseline performance characteristics, grouped by input size. In the first column, † marks benchmarks that report request latency after each iteration. The next two columns show the # of iterations and the minimum heap size in MB. The next three columns show the total time to run the benchmark, CPU utilization as a % of all computing cores (as reported by `time`, out of 1,600%), and the maximum heap usage (in GB) in the baseline configuration with -Xmx32g. The final three columns show the same measurements for the baseline with -Xmx set to 160 GB. All measurements were collected on the Intel Xeon platform.

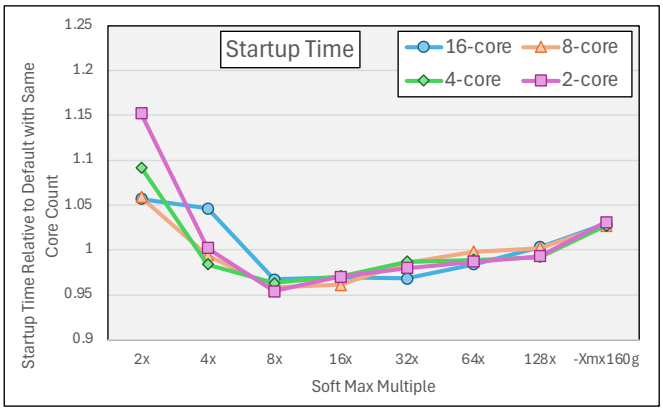
C Supplemental Results

C.1 Intel Xeon Platform

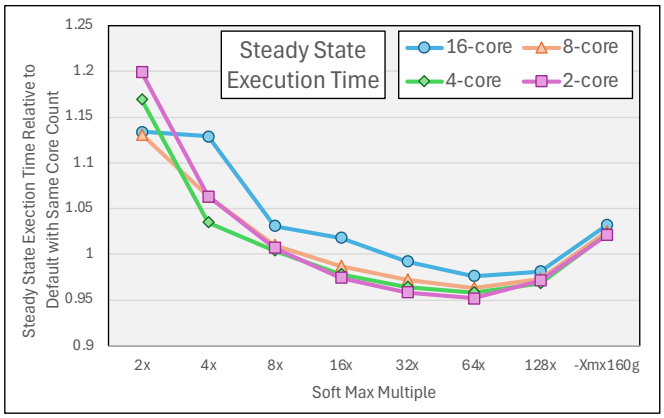
C.1.1 Hard Max and Soft Max with Varying CPU Constraints.



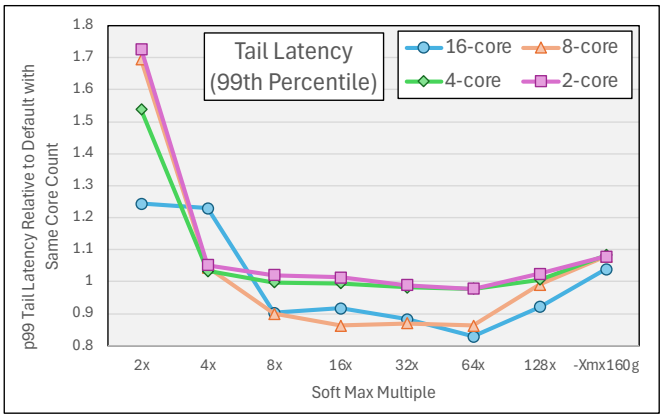
(a) Max heap used (GB, log scale).



(b) Startup time.



(c) Steady-state execution time.



(d) Tail latency (99th percentile).

Figure 9. Average results for the DaCapo benchmarks with `-XX:SoftMaxHeapSize` set to different multiples of the empirically-derived minimum heap size, with 2, 4, 8, or 16 cores available to the JVM. Performance results are shown relative to the default configuration with the same core count.

C.1.2 Per Benchmark Results for Evaluation with 16 Cores.

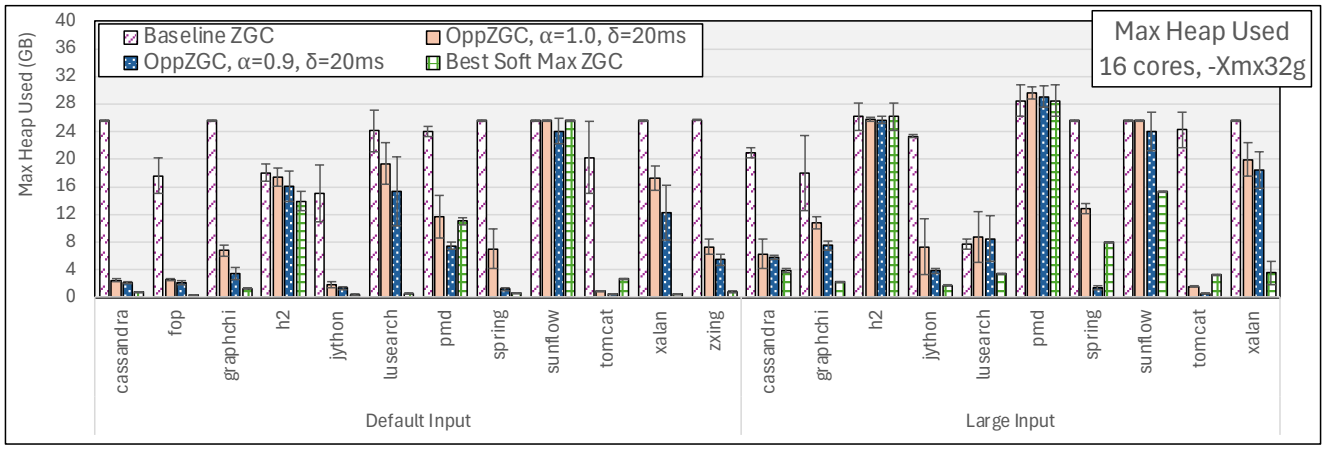


Figure 10. Maximum heap occupancy (GB) for each DaCapo benchmark, 16 cores, `-Xmx32g`.

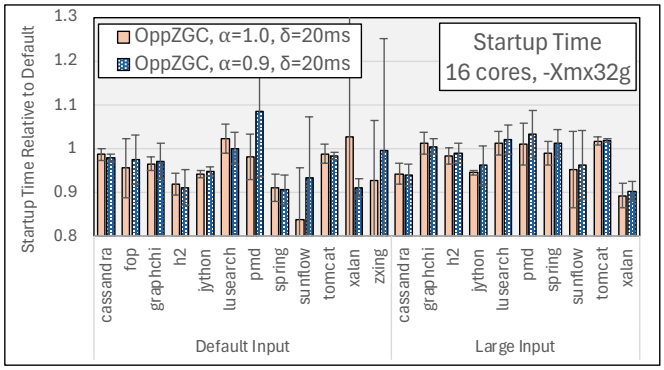


Figure 11. Startup time relative to baseline ZGC, 16 cores, `-Xmx32g`.

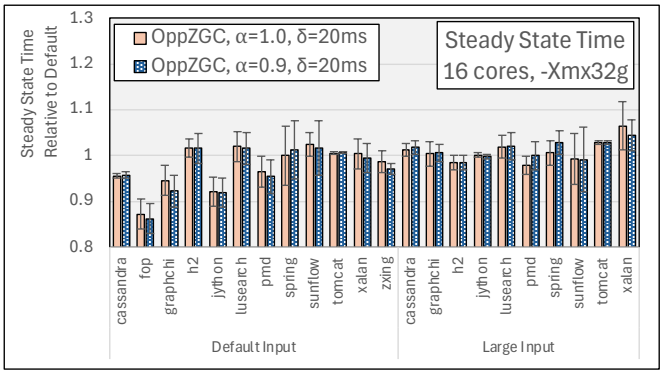


Figure 12. Steady-state time relative to baseline ZGC, 16 cores, `-Xmx32g`.

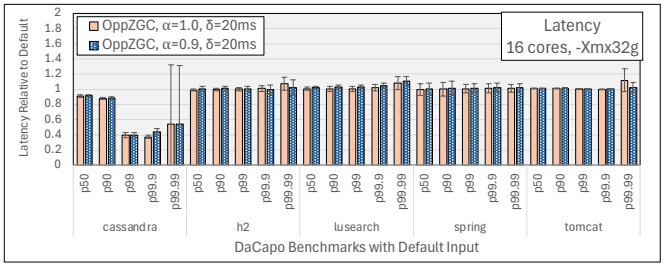


Figure 13. Latency distribution relative to baseline ZGC, default inputs, 16 cores, `-Xmx32g`.

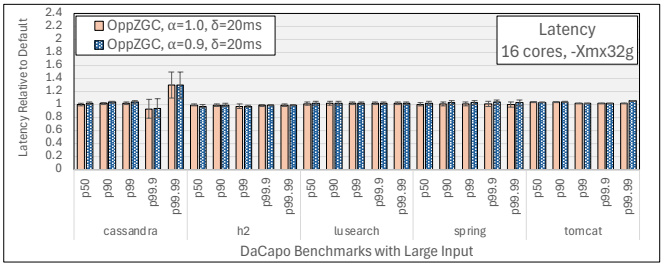


Figure 14. Latency distribution relative to baseline ZGC, large inputs, 16 cores, `-Xmx32g`.

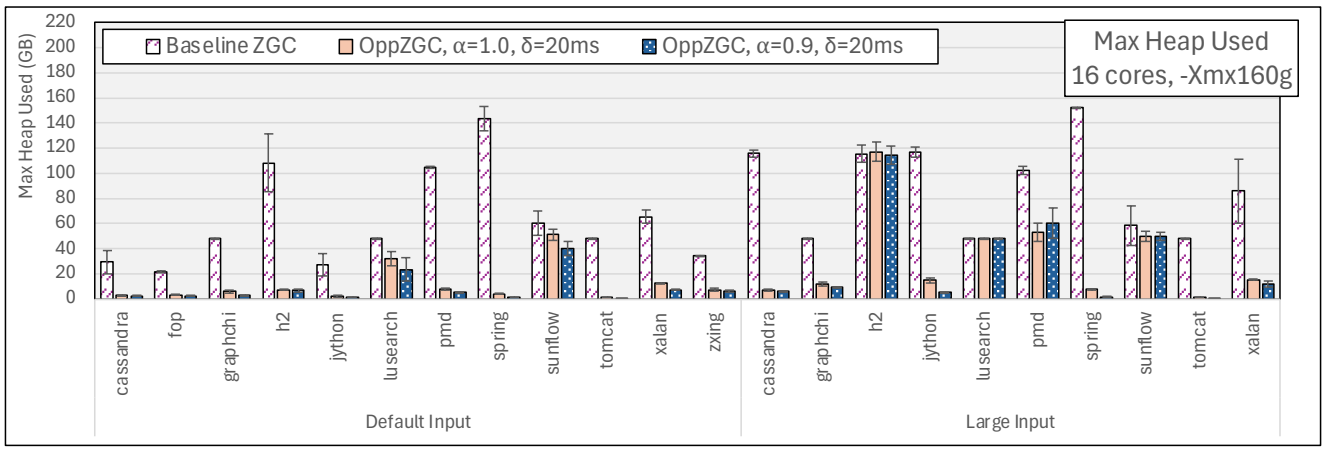


Figure 15. Maximum heap occupancy (GB) for each DaCapo benchmark, 16 cores, `-Xmx160g`.

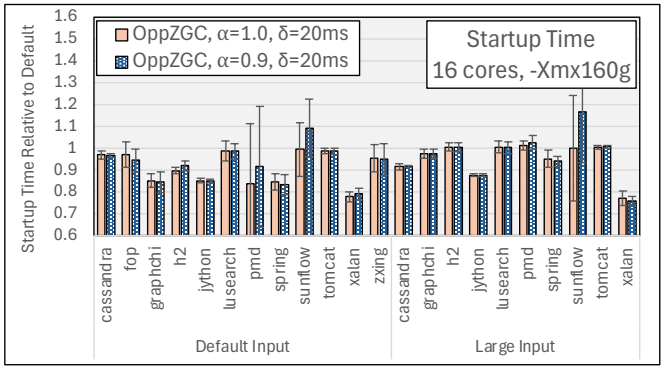


Figure 16. Startup time relative to baseline ZGC, 16 cores, `-Xmx160g`.

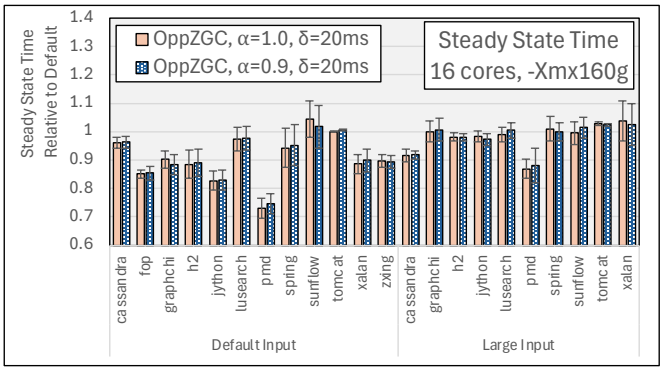


Figure 17. Steady-state time relative to baseline ZGC, 16 cores, `-Xmx160g`.

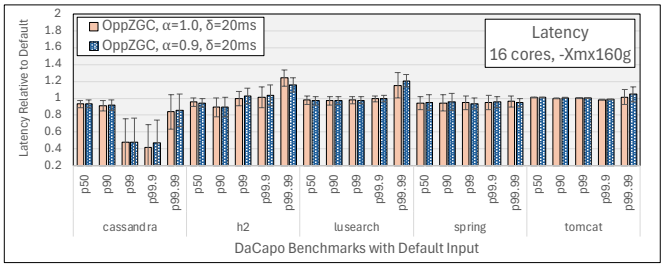


Figure 18. Latency distribution relative to baseline ZGC, default inputs, 16 cores, `-Xmx160g`.

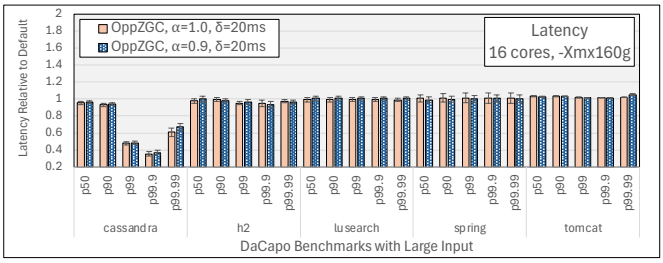


Figure 19. Latency distribution relative to baseline ZGC, large inputs, 16 cores, `-Xmx160g`.

C.1.3 Per Benchmark Results for Evaluation with Varying Cores.

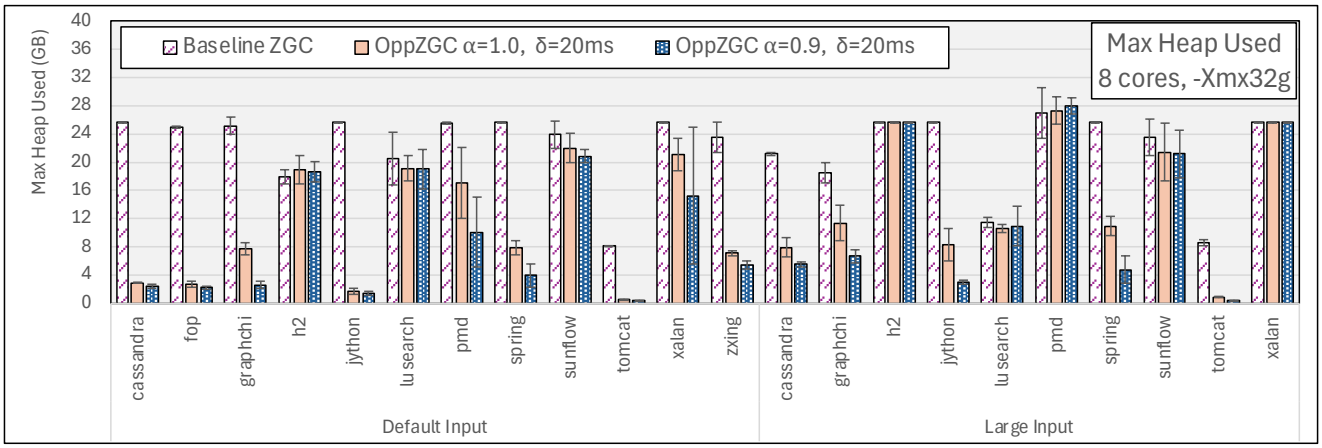


Figure 20. Maximum heap occupancy (GB) for each DaCapo benchmark, 8 cores, `-Xmx32g`.

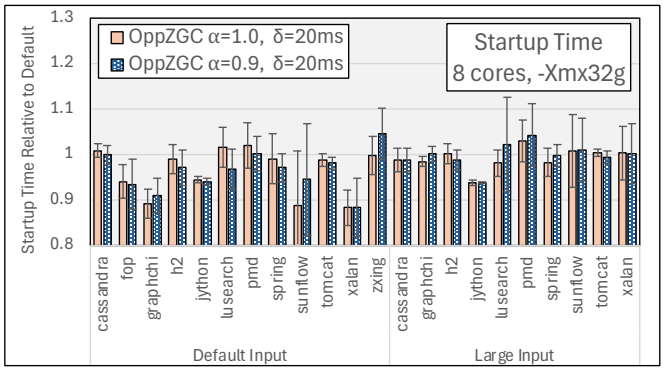


Figure 21. Startup time relative to baseline ZGC, 8 cores, `-Xmx32g`.

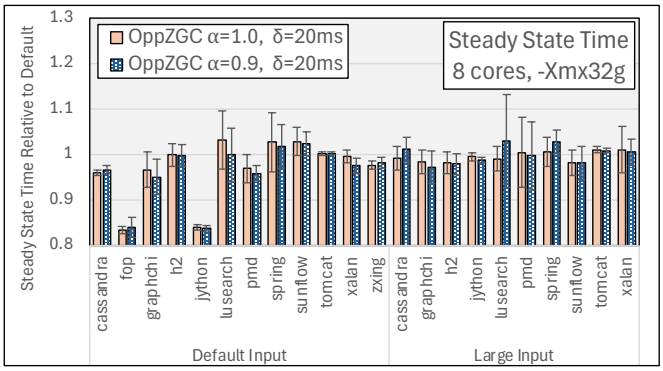


Figure 22. Steady-state time relative to baseline ZGC, 8 cores, `-Xmx32g`.

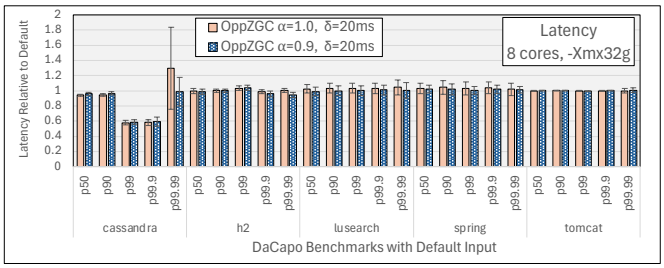


Figure 23. Latency distribution relative to baseline ZGC, default inputs, 8 cores, `-Xmx32g`.

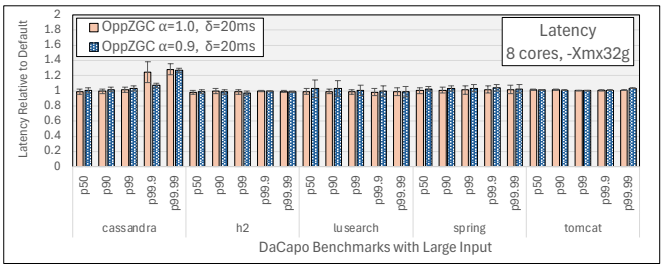


Figure 24. Latency distribution relative to baseline ZGC, large inputs, 8 cores, `-Xmx32g`.

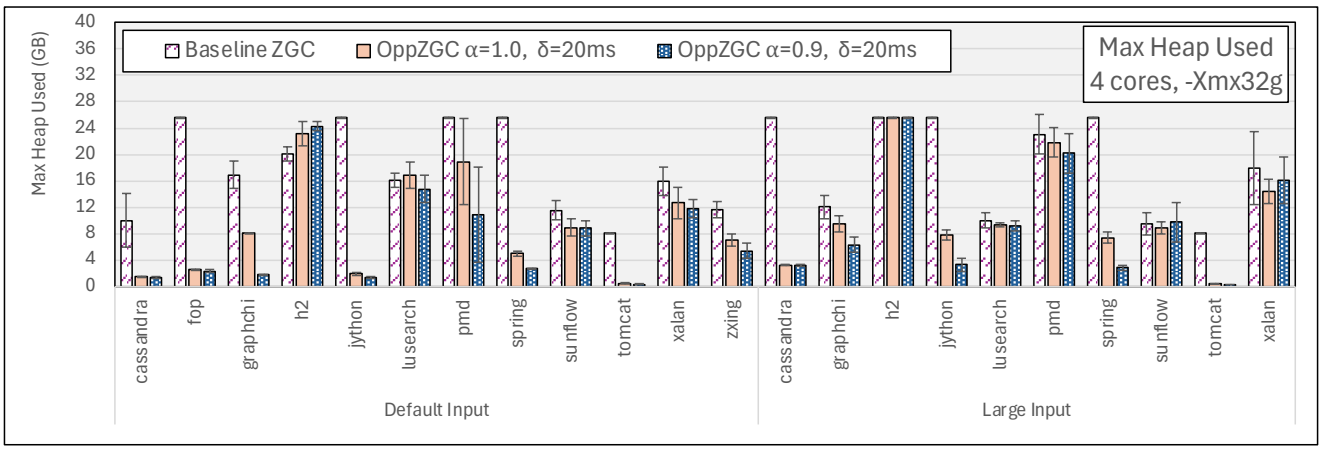


Figure 25. Maximum heap occupancy (GB) for each DaCapo benchmark, 4 cores, `-Xmx32g`.

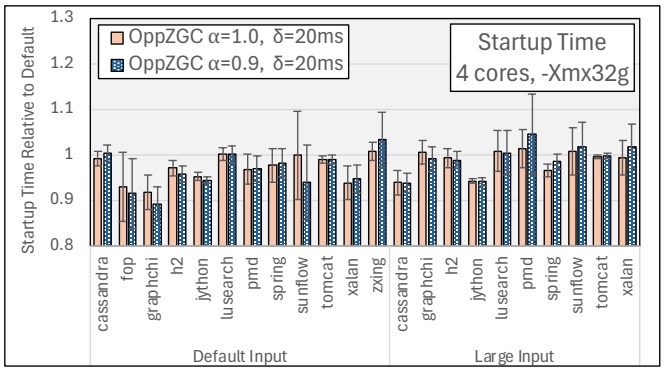


Figure 26. Startup time relative to baseline ZGC, 4 cores, `-Xmx32g`.

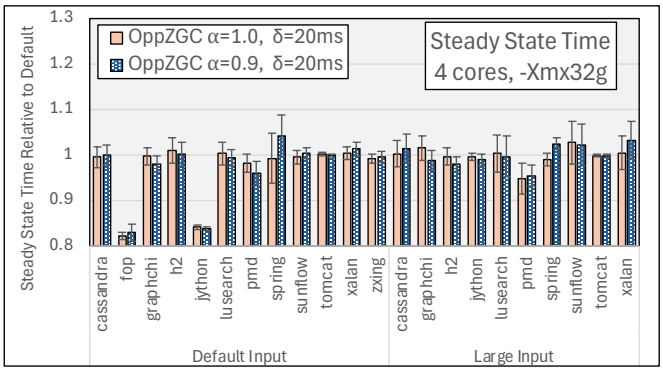


Figure 27. Steady-state time relative to baseline ZGC, 4 cores, `-Xmx32g`.

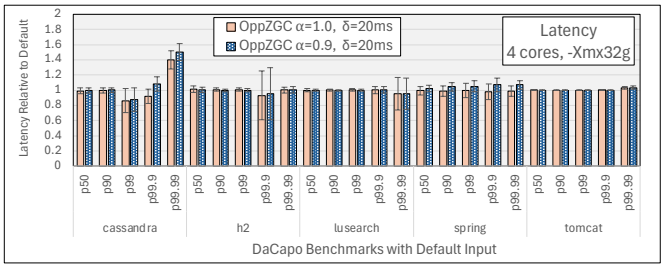


Figure 28. Latency distribution relative to baseline ZGC, default inputs, 4 cores, `-Xmx32g`.

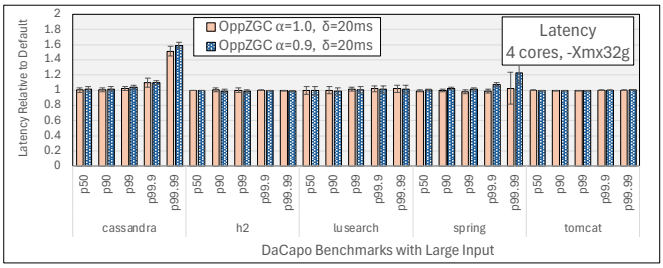


Figure 29. Latency distribution relative to baseline ZGC, large inputs, 4 cores, `-Xmx32g`.

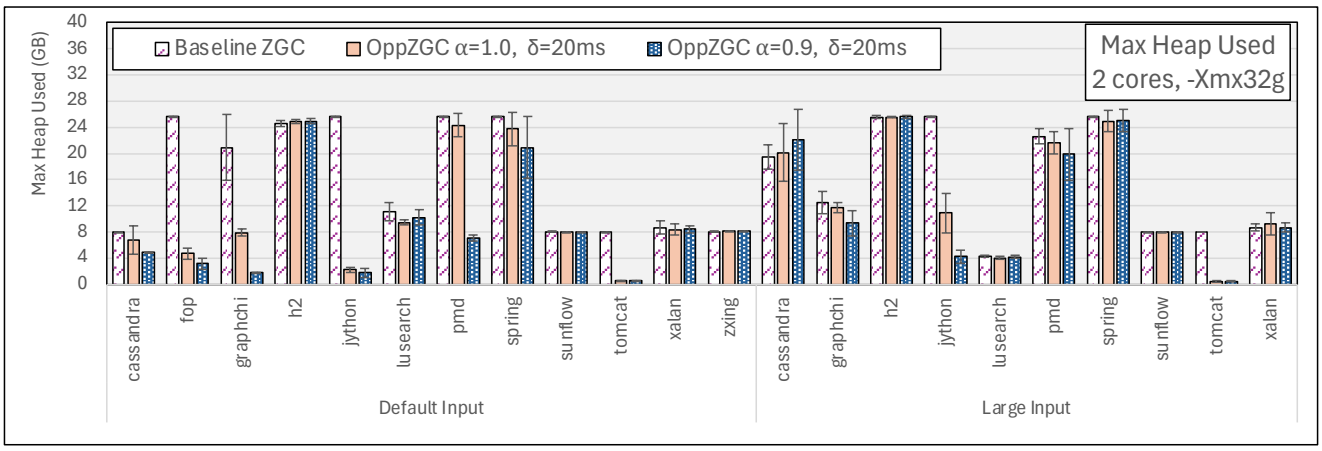


Figure 30. Maximum heap occupancy (GB) for each DaCapo benchmark, 2 cores, `-Xmx32g`.

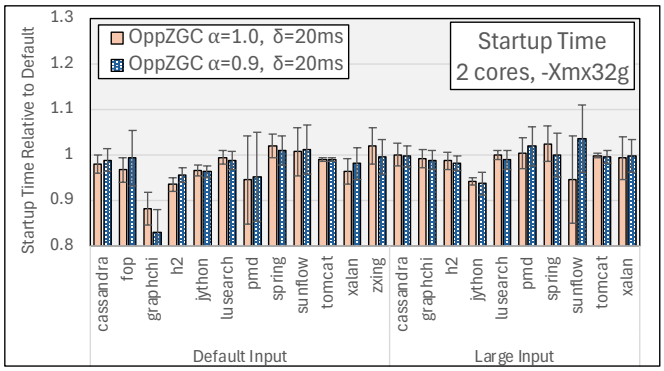


Figure 31. Startup time relative to baseline ZGC, 2 cores, `-Xmx32g`.

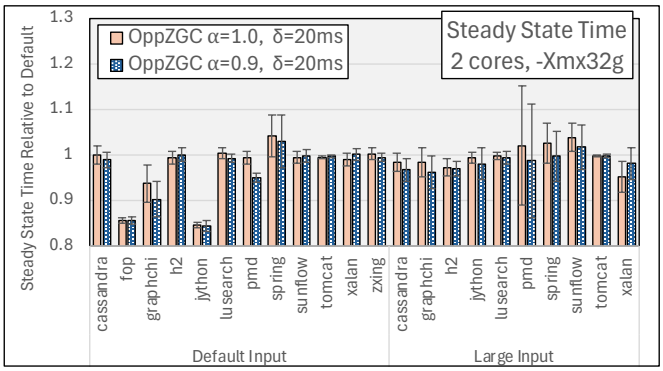


Figure 32. Steady-state time relative to baseline ZGC, 2 cores, `-Xmx32g`.

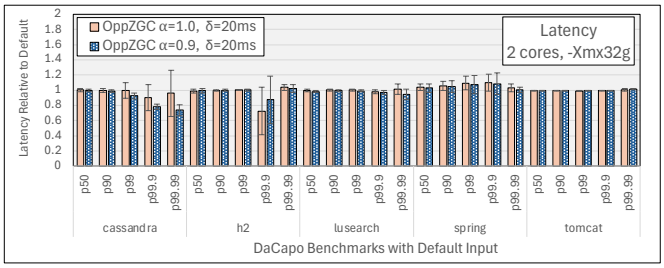


Figure 33. Latency distribution relative to baseline ZGC, default inputs, 2 cores, `-Xmx32g`.

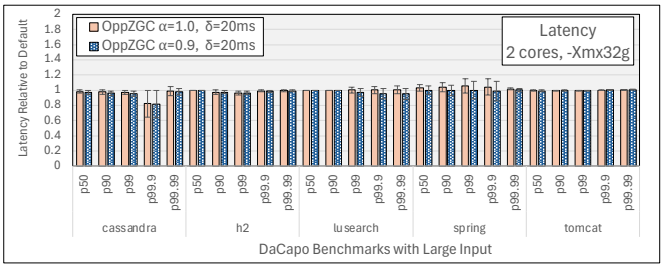


Figure 34. Latency distribution relative to baseline ZGC, large inputs, 2 cores, `-Xmx32g`.

C.2 AMD Platform

C.2.1 Platform Description. Our second experimental platform contains a single AMD Ryzen 9 5950X CPU with 16 physical compute cores and simultaneous multithreading disabled. The cores run at a 3.4 GHz base clock (boost disabled) and share a 64 MB L3 cache, split evenly across two 8-core core complex dies (CCDs), each with its own 32 MB L3 slice. The processor's memory controller services two channels, each connected to two 16 GB, 3600 MT/s, DDR4 DIMMs, for a total of 64 GB of DDR4 SDRAM. As such all reported benchmarks on the AMD platform use `-Xmx32g`.

C.2.2 Experimental Setup. The evaluation on the AMD platform uses the same system and runtime software versions and configuration, as well as the same experimental configuration and parameters, as the evaluation on the Intel platform. For the results in this appendix, we ran our DaCapo default and large input benchmarks and the SPECjbb2015 benchmark with three configurations: 1) baseline ZGC, 2) OppZGC with $\alpha = 1.0$ and $\delta = 20\,ms$, and 3) OppZGC with $\alpha = 0.9$ and $\delta = 20\,ms$. All results reported here use all 16 cores of the platform and a maximum heap size of 32 GB.

C.2.3 Evaluation Notes. The results on the AMD platform show very similar performance and memory usage trends to the evaluation on the Intel platform. On average, OppZGC reduces the maximum heap occupancy for the DaCapo default and large inputs by 59% and 35% with $\alpha = 1.0$, and by 69% and 45% with $\alpha = 0.9$. Additionally, OppZGC exhibits similar startup time, throughput, and request latency to baseline ZGC with the DaCapo benchmarks, in almost all cases. Notably, OppZGC increases p99.99 tail latency for *cassandra* by about 84% to 93% with its large input, and by 36% to 55% with its default input, while leaving p99 and p99.9 at or below baseline. For SPECjbb, we find OppZGC reduces the maximum heap occupancy by 20%, while maintaining the same max jOPS and critical jOPS as baseline ZGC. We are currently collecting core-restricted results for the DaCapo benchmarks on the AMD platform and plan to include these in the final version of this appendix.

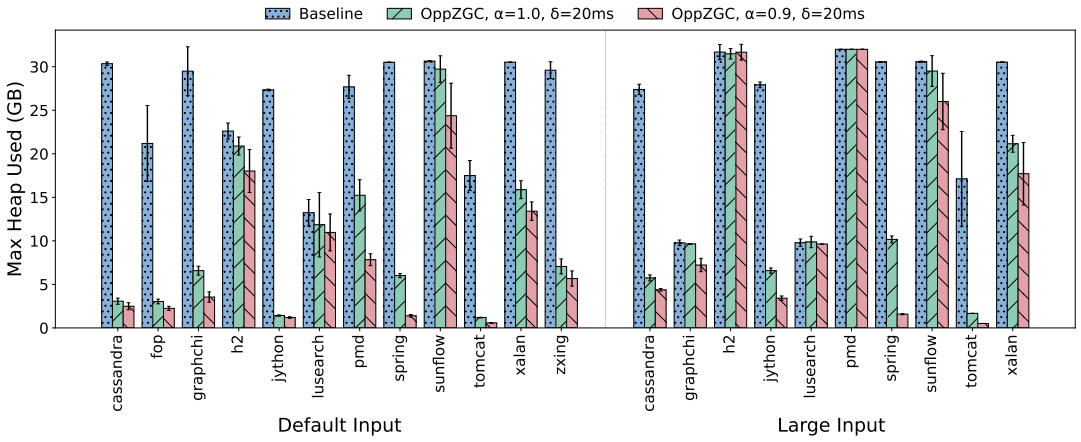


Figure 35. Maximum heap occupancy (in GB) under OppZGC, relative to baseline ZGC.

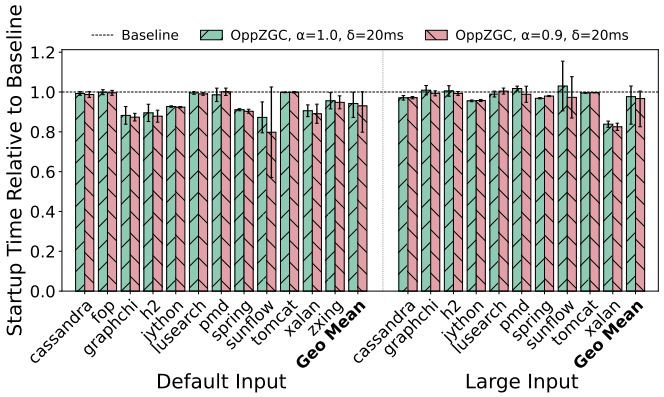


Figure 36. Startup time.

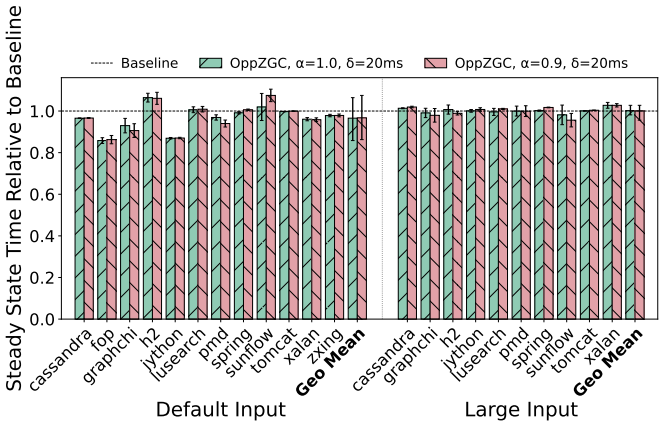


Figure 37. Steady-state time.

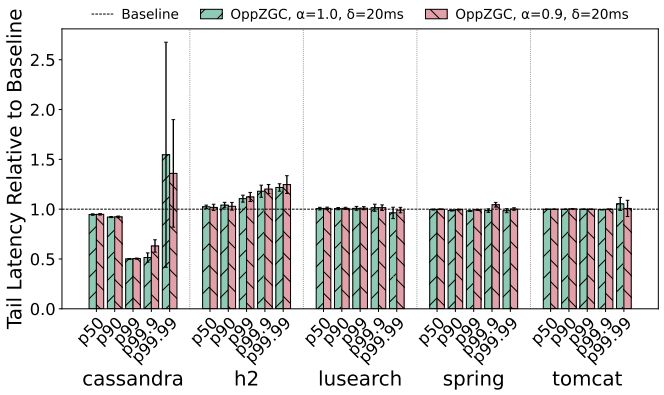


Figure 38. Latency distribution, default input.

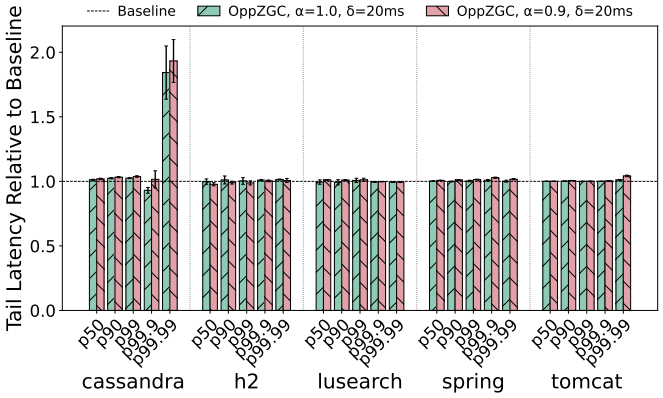


Figure 39. Latency distribution, large input.

C.2.4 SPECjbb2015 Results on the AMD Platform.

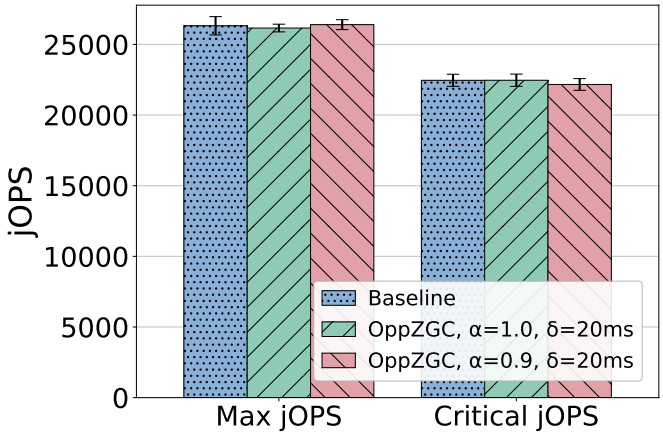


Figure 40. Throughput.

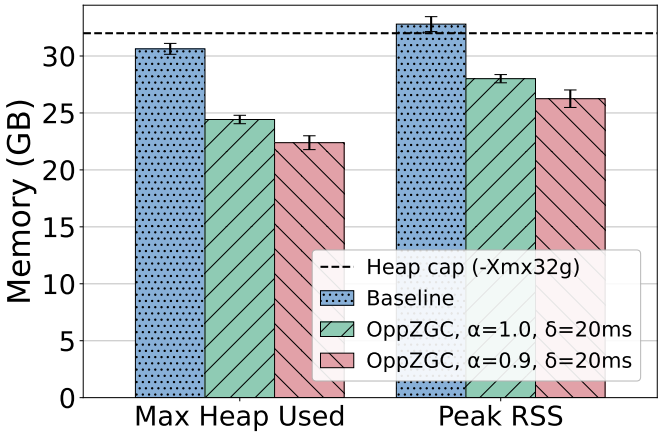


Figure 41. Memory usage.